

INFORMATIQUE • 1ÈRE TI

MODULE 1 : ALGORITHMIQUE

LEÇON N04

ALGORITHMES DE TRI ET DE RECHERCHE

DURÉE CONSEILLÉE : 8 H

DANS CETTE LEÇON

Cette leçon présente les algorithmes fondamentaux de tri (tri par sélection, tri par insertion) et de recherche (recherche séquentielle, recherche dichotomique) sur des tableaux de valeurs numériques. L'élève apprend à exécuter ces algorithmes à la main, à décrire leurs principes et à les écrire en langage algorithmique.

- À la fin de cette leçon, je sais énumérer quelques algorithmes de tri classiques
- À la fin de cette leçon, je sais exécuter pas à pas le tri par sélection sur un tableau de nombres
- À la fin de cette leçon, je sais exécuter pas à pas le tri par insertion sur un tableau de nombres
- À la fin de cette leçon, je sais écrire en langage algorithmique un algorithme de tri simple sur un tableau à valeurs numériques
- À la fin de cette leçon, je sais décrire le principe de la recherche séquentielle dans un tableau
- À la fin de cette leçon, je sais décrire le principe de la recherche dichotomique et sa condition d'application

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. Généralités sur le tri et la recherche | rithme |
| 2. Le tri par sélection | 7. Activité d'intégration |
| 3. Le tri par insertion | 8. Méthodes et exercices résolus |
| 4. La recherche séquentielle | 9. Exercices |
| 5. La recherche dichotomique | 10. Corrigés des exercices |
| 6. Synthèse : choisir et écrire le bon algo- | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais énumérer quelques algorithmes de tri classiques
- À la fin de cette leçon, je sais exécuter pas à pas le tri par sélection sur un tableau de nombres
- À la fin de cette leçon, je sais exécuter pas à pas le tri par insertion sur un tableau de nombres
- À la fin de cette leçon, je sais écrire en langage algorithmique un algorithme de tri simple sur un tableau à valeurs numériques
- À la fin de cette leçon, je sais décrire le principe de la recherche séquentielle dans un tableau
- À la fin de cette leçon, je sais décrire le principe de la recherche dichotomique et sa condition d'application
- À la fin de cette leçon, je sais écrire un algorithme permettant de rechercher un élément dans un tableau
- À la fin de cette leçon, je sais exécuter à la main un algorithme de recherche dichotomique sur un tableau trié

PRÉREQUIS

- Notion d'algorithme et de langage algorithmique (variables, affectation)
- Structures de contrôle : si...alors...sinon, boucles pour et tant que
- Notion de tableau à une dimension : déclaration, indices, accès à un élément

- Fonctions prédéfinies simples (longueur d'un tableau, division entière)

1 GÉNÉRALITÉS SUR LE TRI ET LA RECHERCHE

La bibliothécaire du lycée de Ngaoundéré vient de recevoir 500 nouveaux manuels qu'elle doit ranger par ordre de cote : comment s'y prendre pour finir avant la fin de la semaine ? De son côté, le gérant d'un kiosque à Yaoundé cherche le numéro d'un client dans un cahier de 300 pages : doit-il feuilleter page par page ? Et si le cahier était trié par ordre alphabétique, ne pourrait-il pas ouvrir directement au milieu et éliminer la moitié des pages d'un coup ? Ces situations quotidiennes cachent deux familles d'algorithmes fondamentaux en informatique : les **algorithmes de tri** et les **algorithmes de recherche**. Cette leçon nous apprend à les décrire, les exécuter et les écrire.

Problématique : comment organiser efficacement un ensemble de données, et comment retrouver rapidement une information dans cet ensemble ?

1.1 Pourquoi trier ? Pourquoi rechercher ?

Avant toute formule, posons-nous une question simple : à quoi servent ces algorithmes dans la vie réelle ?

- Le proviseur veut publier le **classement des notes** du conseil de classe : il faut trier les moyennes de la meilleure à la moins bonne.
- L'opérateur téléphonique maintient un **annuaire** de millions d'abonnés : pour facturer un appel, il faut rechercher un numéro en une fraction de seconde.
- Le secrétariat du lycée gère le **fichier des élèves** : trouver le dossier de « MBALLA Aïcha » parmi 1 200 élèves exige une méthode de recherche, et ce dossier est d'autant plus facile à trouver que le fichier est rangé par ordre alphabétique.
- La boutique de quartier tient un stock : les articles sont triés par référence pour être retrouvés instantanément.

Deux constats s'imposent. D'abord, les données désordonnées sont presque inutilisables : personne ne consulte un annuaire dont les noms seraient mélangés au hasard. Ensuite, tri et recherche sont intimement liés : on trie souvent pour pouvoir rechercher vite ensuite. C'est exactement la stratégie du gérant du kiosque : trier son cahier une fois, puis gagner du temps à chaque recherche.

1.2 Définition du tri d'un tableau

Intuitivement, trier consiste à ranger. En algorithmique, les données à ranger sont stockées dans un **tableau**, et le rangement porte sur une relation d'ordre : l'ordre croissant ou décroissant sur des nombres, l'ordre alphabétique sur des chaînes de caractères.

DÉFINITION Tri d'un tableau

Trier un tableau, c'est réorganiser ses éléments de manière qu'ils se succèdent selon un ordre donné :

- ordre croissant : chaque élément est inférieur ou égal au suivant : $T[1] \leq T[2] \leq \dots \leq T[n]$;
- ordre décroissant : chaque élément est supérieur ou égal au suivant : $T[1] \geq T[2] \geq \dots \geq T[n]$.

Le tri ne change ni le nombre d'éléments, ni leurs valeurs : il ne fait que permuer (échanger) les éléments à l'intérieur du tableau.

Tableau désordonné

12	4	9	1	7
0	1	2	3	4

trier →

Tableau trié (ordre croissant)

1	4	7	9	12
0	1	2	3	4

Effet d'un tri croissant sur le tableau [12; 4; 9; 1; 7] : les mêmes cinq valeurs, rangées dans l'ordre [1; 4; 7; 9; 12]. Les indices (0 à 4) désignent les positions dans le tableau.

ATTENTION Position (indice) ou valeur : ne pas confondre !

Dans le tableau trié ci-dessus, la valeur 7 se trouve à la position (indice) 2. Ce sont deux notions différentes :

- la **valeur** est le contenu de la case (ici 7) ;
- la **position** (ou indice) est le numéro de la case (ici 2, si l'on numérote à partir de 0).

Erreur classique au Probatoire : un algorithme de recherche doit retourner la position de l'élément trouvé, pas sa valeur (la valeur, on la connaît déjà, puisque c'est ce qu'on cherche!). Autre piège : selon les langages, le premier indice est 0 (C, JavaScript) ou 1 (pseudo-code MINESEC, parfois). Dans cette leçon, en pseudo-code, nous numérote-rons les cases de 1 à n .

1.3 Définition de la recherche d'un élément

Une fois les données organisées (ou non), se pose la question du gérant du kiosque : un élément donné figure-t-il dans le tableau, et où ?

DÉFINITION Recherche d'un élément dans un tableau

Rechercher un élément x dans un tableau T de n éléments, c'est déterminer :

1. si x appartient au tableau, c'est-à-dire s'il existe une position i telle que $T[i] = x$;
2. et, dans l'affirmative, à quelle position i il se trouve.

Si l'élément est absent, l'algorithme doit le signaler clairement (par exemple en retournant la position conventionnelle 0 ou -1, ou un booléen « non trouvé »).

EXEMPLE Recherche dans le fichier des élèves

Le secrétariat dispose du tableau de noms : [ETOA, MBALLA, NGONO, TCHOUP0]. Rechercher NGONO doit répondre : « trouvé à la position 3 ». Rechercher KAMGA doit répondre : « absent du tableau ». Remarque : ici le tableau est trié par ordre alphabétique, ce qui permettra (section 5) une recherche beaucoup plus rapide que le feuilletage page par page.

1.4 Énumération de quelques algorithmes de tri

Il n'existe pas une seule méthode de tri, mais des dizaines. Les informaticiens les ont inventées au fil des décennies, chacune avec ses avantages. En voici quatre qu'il faut savoir énumérer (savoir-faire officiel) :

Algorithme	Principe en une phrase
Tri par sélection	On cherche le plus petit élément, on le place en première position, puis on recommence avec le reste du tableau.
Tri par insertion	On prend les éléments un par un et on les insère à leur place dans la partie déjà triée (comme on trie ses cartes à la main).
Tri à bulles	On compare les éléments voisins deux à deux et on les échange s'ils sont dans le mauvais ordre ; les plus grands « remontent » comme des bulles.
Tri rapide (quicksort)	On choisit un élément « pivot », on place les plus petits à sa gauche et les plus grands à sa droite, puis on recommence sur chaque moitié.

Au programme de Première TI, seuls le **tri par sélection** et le **tri par insertion** sont étudiés et exécutés en détail (sections 2 et 3). Le tri à bulles et le tri rapide sont simplement cités, pour la culture algorithmique.

LESAVAIS-TU ? Le tri rapide, champion du monde

Le **tri rapide** (quicksort) a été inventé en 1960 par l'informaticien britannique Tony Hoare, alors qu'il travaillait sur un projet de traduction automatique. Soixante ans plus tard, il reste l'un des algorithmes les plus utilisés au monde : il est au cœur des bibliothèques de tri de nombreux langages de programmation. Son idée géniale — diviser le problème en deux sous-problèmes plus petits autour d'un « pivot » — illustre la stratégie « diviser pour régner », que tu retrouveras dans la recherche dichotomique. Tony Hoare a reçu en 1980 le prix Turing, l'équivalent du prix Nobel d'informatique.

1.5 La notion de coût d'un algorithme

Revenons à la bibliothécaire de Ngaoundéré et à ses 500 manuels. Plusieurs méthodes de tri existent : laquelle choisir ? Pour comparer deux algorithmes qui résolvent le même problème, on évalue leur **coût**, c'est-à-dire la quantité de travail qu'ils demandent.

DÉFINITION Coût d'un algorithme de tri ou de recherche (approche intuitive)

Le **coût** d'un algorithme se mesure, à notre niveau, en comptant les opérations élémentaires qu'il effectue :

- le nombre de comparaisons entre éléments (ex. « $T[i] < T[j]$? ») ;
- le nombre d'échanges (ou déplacements/permutations) d'éléments dans le tableau.

Plus ces nombres sont grands, plus l'algorithme est lent, surtout quand le tableau contient beaucoup d'éléments.

Prenons une intuition chiffrée. Pour un tableau de n éléments :

- la **recherche séquentielle** (feuilleter page par page) peut exiger jusqu'à n comparaisons dans le pire des cas : pour 300 pages, jusqu'à 300 regards ;
- la **recherche dichotomique** (ouvrir au milieu, éliminer la moitié) divise la zone de recherche par 2 à chaque étape : pour 300 pages, environ 9 étapes suffisent, car $2^9 = 512 \geq 300$.

L'écart est spectaculaire : 300 contre 9 ! De même, les tris « simples » (sélection, insertion, bulles) effectuent de l'ordre de $n \times n$ comparaisons — soit 250 000 pour 500 manuels — tandis que le tri rapide en effectue de l'ordre de $n \times \log_2 n$, soit environ 4 500. C'est pourquoi le choix de l'algorithme n'est pas un luxe : c'est la différence entre finir vendredi soir... ou le mois prochain.

À RETENIR Ce qu'il faut retenir sur le coût

À ce stade, nul besoin de formules savantes : retiens simplement qu'un bon algorithme fait moins de comparaisons et moins d'échanges, et que l'écart devient énorme quand le nombre n de données grandit. Trier une fois un tableau pour ensuite y faire des recherches dichotomiques est souvent la stratégie gagnante.

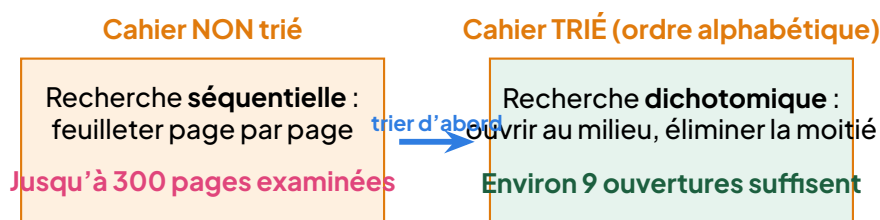
1.6 Le lien entre tri et recherche

La recherche dichotomique — la méthode du gérant qui ouvre son cahier au milieu — possède une condition d'application absolue :

PROPRIÉTÉ Condition d'application de la recherche dichotomique

La **recherche dichotomique** ne peut être appliquée que sur un tableau préalablement trié (par exemple en ordre croissant). Si le tableau n'est pas trié, le fait d'éliminer la moitié des éléments après comparaison avec l'élément du milieu n'a aucune garantie de validité : l'élément cherché pourrait se trouver dans la moitié éliminée.

C'est toute la morale de notre histoire : le gérant du kiosque ne peut ouvrir son cahier au milieu et éliminer 150 pages d'un coup que parce que les noms y sont rangés par ordre alphabétique. Le tri prépare le terrain ; la recherche en récolte les fruits.



Rechercher « NGONO » dans un cahier de 300 pages : sans tri, on feuillette page par page ; avec un cahier trié, chaque ouverture au milieu élimine la moitié des pages restantes.

1.7 Premier exemple guidé : observer avant d'algorithmiser

Avant d'écrire le moindre algorithme, entraînons l'œil sur un petit tableau, comme le ferait un programmeur qui teste son idée « à la main ».

EXERCICE RÉSOLU 1 Tri et recherche sur un mini-tableau

Soit le tableau $T = [15; 3; 9; 3; 12]$ de $n = 5$ éléments (indices de 1 à 5).

1. Donner le contenu de T après un tri par ordre croissant.
2. On recherche la valeur 9 dans le tableau trié. Que doit retourner l'algorithme de recherche ?
3. On recherche la valeur 7. Que doit-il retourner ?
4. Combien de comparaisons, au maximum, exige une recherche séquentielle dans ce tableau ? Et une recherche dichotomique ?

SOLUTION

Solution détaillée.

1. On range les valeurs de la plus petite à la plus grande : $T = [3; 3; 9; 12; 15]$. Remarque : la valeur 3 apparaît deux fois ; le tri conserve tous les éléments, doublons compris.

2. La valeur 9 se trouve à la position 3 : l'algorithme retourne 3 (la position, pas la valeur!).
3. La valeur 7 n'apparaît pas : l'algorithme retourne un signal d'absence, par exemple 0 (ou « non trouvé »).
4. Recherche séquentielle : dans le pire des cas, on examine les 5 cases, donc au maximum 5 comparaisons. Recherche dichotomique : à chaque étape, on divise la zone par 2 : $5 \rightarrow 3 \rightarrow 1$, soit au maximum 3 comparaisons.

Conclusion : même sur un tableau minuscule, la dichotomie gagne ; sur 300 pages ou 500 manuels, l'écart devient décisif.

EXERCICE 1 À toi de jouer

Le tableau des moyennes d'une classe est $M = [11,5 ; 8 ; 14,5 ; 10 ; 8]$.

1. Écris le tableau trié par ordre décroissant (du major au dernier).
2. On recherche la moyenne 10 dans le tableau trié : quelle position l'algorithme doit-il retourner ?
3. Pourquoi ne peut-on pas appliquer la recherche dichotomique sur le tableau M avant de l'avoir trié ?

CORRIGÉ Exercice : moyennes de la classe

1. Ordre décroissant : $M = [14,5 ; 11,5 ; 10 ; 8 ; 8]$.
2. La valeur 10 est à la position 3 dans le tableau trié.
3. Parce que la dichotomie repose sur l'élimination d'une moitié du tableau après comparaison avec l'élément central : sans ordre préalable, rien ne garantit que la valeur cherchée n'est pas dans la moitié éliminée. La recherche dichotomique exige un tableau trié.

À RETENIR Bilan de la section

- Trier un tableau, c'est réorganiser ses éléments selon un ordre (croissant ou décroissant), sans ajouter ni supprimer de valeur.
- Rechercher, c'est déterminer si un élément appartient au tableau et retourner sa position (jamais sa valeur).
- Quelques tris à savoir énumérer : sélection, insertion, à bulles, rapide.

- Le coût se mesure en nombre de comparaisons et d'échanges : il explique pourquoi certains algorithmes écrasent les autres dès que les données sont nombreuses.
- Tri et recherche sont liés : la recherche dichotomique exige un tableau trié.

Nous sommes maintenant prêts à entrer dans le détail du premier algorithme de tri du programme : le tri par sélection, objet de la section suivante.

2 LE TRI PAR SÉLECTION

Dans la section précédente, nous avons vu ce qu'est un **tri** : réorganiser les éléments d'un tableau dans un ordre croissant (ou décroissant), et nous avons distingué les deux grandes familles d'algorithmes que nous allons étudier : les algorithmes de **tri** et les algorithmes de **recherche**. Nous attaquons maintenant le premier algorithme de tri du programme : le **tri par sélection**. C'est l'un des tris les plus simples à comprendre et à exécuter à la main, ce qui en fait un excellent point de départ.

2.1 Intuition et principe

Une situation concrète. Imagine le professeur de TI qui range les copies du devoir de la classe de Première TI par ordre croissant de notes, étalées sur la table. Une stratégie très naturelle consiste à faire ceci : parcourir toutes les copies, repérer la plus petite note, et placer cette copie en première position. Puis, parmi les copies restantes, repérer à nouveau la plus petite note et la placer en deuxième position. Et ainsi de suite, jusqu'à ce qu'il ne reste plus qu'une copie — forcément la plus grande — qui prend la dernière place. C'est exactement l'idée du tri par sélection : à chaque étape, on sélectionne le plus petit élément de ce qui reste à trier, et on le met à sa place définitive.

DÉFINITION Tri par sélection

Le **tri par sélection** est un algorithme de tri qui consiste, pour un tableau de n éléments, à répéter l'opération suivante : rechercher le plus petit élément de la partie non encore triée du tableau, puis l'échanger avec le premier élément de cette partie non triée. Après chaque passage, la partie triée s'agrandit d'un élément.

La notion de partie triée et de partie non triée. À tout moment de l'exécution, le tableau est mentalement découpé en deux zones :

- la **partie triée**, située à **gauche** : elle contient les plus petits éléments, déjà rangés dans l'ordre croissant, et on n'y touchera plus jamais ;
- la **partie non triée**, située à **droite** : elle contient les éléments qu'il reste à ranger.

Au départ, la partie triée est vide et la partie non triée occupe tout le tableau. À la fin, la partie

triée occupe tout le tableau et la partie non triée est vide : le tableau est trié. À chaque passage, la frontière entre les deux zones avance d'une case vers la droite.

À RETENIR L'invariant du tri par sélection

Après le passage numéro i , les i premières cases du tableau contiennent les i plus petits éléments, rangés dans l'ordre croissant, à leur place définitive. Cette propriété, vraie à chaque étape, s'appelle un invariant de boucle : c'est elle qui garantit que l'algorithme est correct.

2.2 Exécution guidée pas à pas

Appliquons le principe sur un exemple complet, le tableau $[12 ; 4 ; 9 ; 1 ; 7]$, qui contient $n = 5$ éléments. Nous allons détailler chaque passage.

MÉTHODE Exécuter un tri par sélection à la main

Pour exécuter un tri par sélection sur papier, procède ainsi :

1. Numérote les cases du tableau (indices de 1 à n , ou de 0 à $n-1$ selon la convention choisie — ici nous utilisons les indices 1 à n).
2. Pour le passage numéro i (en partant de $i = 1$) :
 - (a) considère la partie non triée, c'est-à-dire les cases d'indices i à n ;
 - (b) parcours-la de gauche à droite en mémorisant l'indice du plus petit élément rencontré (pas seulement sa valeur!) ;
 - (c) échange le contenu de la case d'indice i avec le contenu de la case du minimum trouvé.
3. Arrête-toi après le passage $n - 1$: le dernier élément est automatiquement à sa place, il est inutile de le traiter.
4. À chaque étape, réécris l'état complet du tableau en soulignant ou en coloriant la partie triée : cela te permet de vérifier ton travail.

EXEMPLE Tri complet de $[12 ; 4 ; 9 ; 1 ; 7]$ en 4 passages

État initial : $[12 ; 4 ; 9 ; 1 ; 7]$. Partie triée : vide.

Passage 1 (partie non triée : cases 1 à 5). Le minimum de $[12 ; 4 ; 9 ; 1 ; 7]$ est 1, à l'indice 4. On échange les cases 1 et 4 :

$[1 ; 4 ; 9 ; 12 ; 7]$

La partie triée est $[1]$.

Passage 2 (partie non triée : cases 2 à 5, soit [4 ; 9 ; 12 ; 7]). Le minimum est 4, à l'indice 2 : il est déjà en tête de la partie non triée, l'échange ne change rien :

[1 ; 4 ; 9 ; 12 ; 7]

La partie triée est [1 ; 4].

Passage 3 (partie non triée : cases 3 à 5, soit [9 ; 12 ; 7]). Le minimum est 7, à l'indice 5. On échange les cases 3 et 5 :

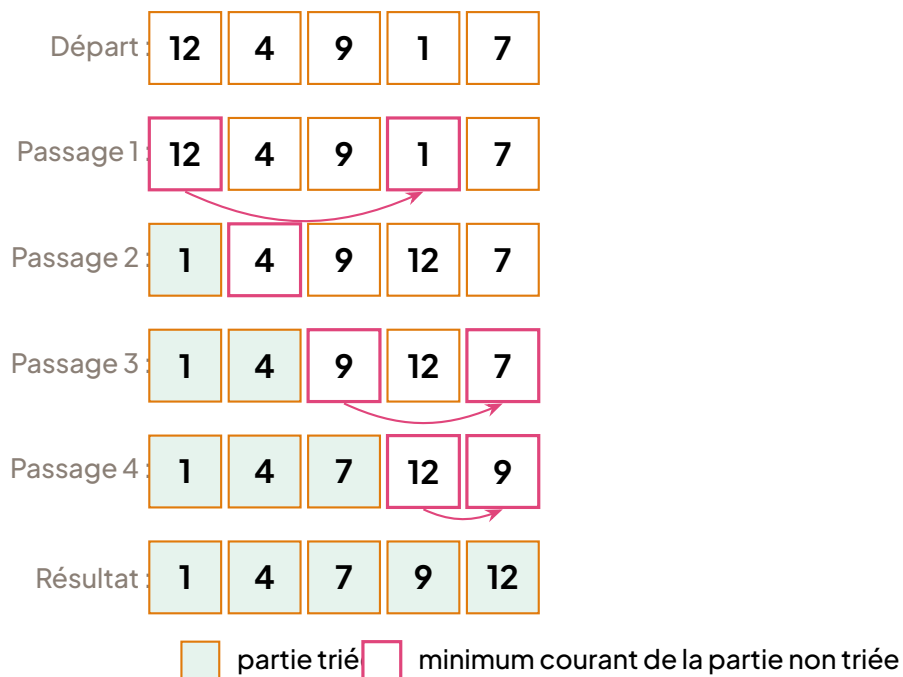
[1 ; 4 ; 7 ; 12 ; 9]

La partie triée est [1 ; 4 ; 7].

Passage 4 (partie non triée : cases 4 à 5, soit [12 ; 9]). Le minimum est 9, à l'indice 5. On échange les cases 4 et 5 :

[1 ; 4 ; 7 ; 9 ; 12]

Il reste une seule case non triée, qui contient forcément le plus grand élément : le tableau est trié : [1 ; 4 ; 7 ; 9 ; 12]. Nous avons effectué exactement $n - 1 = 4$ passages.



États successifs du tableau [12 ; 4 ; 9 ; 1 ; 7] pendant le tri par sélection. À chaque passage, le minimum de la partie non triée (encadré en rouge) est échangé avec le premier élément de cette partie (flèche rouge).

PROPRIÉTÉ Nombre de passages

Pour un tableau de n éléments, le tri par sélection effectue exactement $n - 1$ passages. En effet, une fois que les $n - 1$ plus petits éléments sont placés, le dernier élément est nécessairement le plus grand et occupe déjà la dernière case : un n -ième passage serait inutile.

2.3 L'échange de deux éléments : le rôle de la variable intermédiaire

Le cœur du tri par sélection est l'**échange** (en anglais swap) du contenu de deux cases du tableau. Cette opération semble banale, mais elle cache un piège classique. Pour échanger les contenus de deux variables a et b , il faut obligatoirement passer par une troisième variable, appelée **variable intermédiaire** ou **variable temporaire** (souvent nommée `temp`).

ATTENTION Erreur fréquente : échanger sans variable intermédiaire

Écrire simplement :

```
a <- b  
b <- a
```

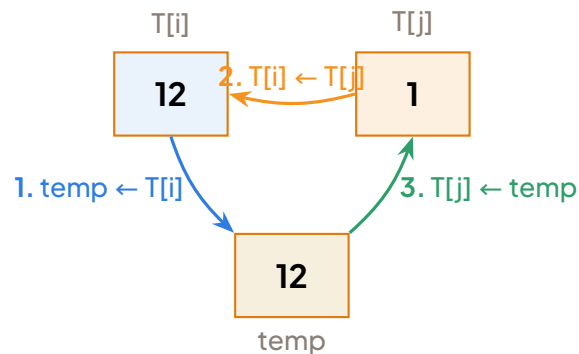
est faux. Après la première ligne, a et b contiennent toutes deux l'ancienne valeur de b : la valeur initiale de a a été écrasée et est définitivement perdue. La deuxième ligne recopie donc dans b la valeur de b elle-même. Résultat : les deux cases contiennent la même valeur, et une donnée du tableau a disparu. C'est l'un des bugs les plus fréquents chez les débutants — et l'un des pièges préférés du Probatoire.

PROPRIÉTÉ Syntaxe de l'échange en trois affectations

Pour échanger les contenus de deux cases $T[i]$ et $T[j]$ d'un tableau, on utilise une variable intermédiaire `temp` :

```
temp <- T[i]  
T[i] <- T[j]  
T[j] <- temp
```

L'ordre des trois affectations est essentiel : on commence toujours par sauvegarder l'une des deux valeurs dans `temp` avant tout écrasement.



Échange des cases $T[i]$ (valeur 12) et $T[j]$ (valeur 1) en trois affectations : on sauvegarde d'abord 12 dans $temp$, puis on écrase $T[i]$ avec 1, puis on restaure 12 dans $T[j]$ grâce à $temp$.

2.4 Algorithme complet du tri par sélection

Nous pouvons maintenant formaliser l'algorithme. Il est constitué de **deux boucles imbriquées** :

- la **boucle extérieure** (indice i) repère le début de la partie non triée : elle va de 1 à $n - 1$ (les $n - 1$ passages) ;
- la **boucle intérieure** (indice j) parcourt la partie non triée, de $i + 1$ à n , pour trouver l'indice du minimum ;
- après la boucle intérieure, on échange $T[i]$ et $T[indiceMin]$ à l'aide de la variable temporaire.

À RETENIR Algorithme du tri par sélection

```

Algorithme Tri_Selection
Variable
  T : Tableau[1..n] de Reel
  i, j, indiceMin : Entier
  temp : Reel
Debut
  Pour i Allant de 1 à n - 1 Faire
    // On suppose que le minimum est en position i
    indiceMin ← i
    // Recherche du minimum dans la partie non triée
    Pour j Allant de i + 1 à n Faire
      Si  $T[j] < T[indiceMin]$  Alors
        indiceMin ← j
      FinSi
    FinPour
    // Échange de  $T[i]$  et de  $T[indiceMin]$ 
    temp ←  $T[i]$ 
     $T[i] \leftarrow T[indiceMin]$ 
     $T[indiceMin] \leftarrow temp$ 

```

FinPour
Fin

Remarque bien que la boucle intérieure démarre à $i+1$: la case i est déjà mémorisée comme minimum provisoire, il est inutile de la comparer avec elle-même. Remarque aussi que l'échange se fait **une seule fois par passage**, après la boucle intérieure, et non à chaque comparaison : c'est ce qui distingue le tri par sélection d'autres tris plus « bavards » en échanges.

2.5 Trace d'exécution

Pour vérifier un algorithme ou répondre à une question d'examen du type « exécuter l'algorithme sur le tableau suivant », on construit une **trace d'exécution** : un tableau donnant, pour chaque passage, les valeurs des variables importantes et l'état du tableau.

Passage i	j parcourt	indiceMin	Échange ?	État du tableau après le passage
1	2 à 5	4 (valeur 1)	$T[1] \leftrightarrow T[4]$	[1 ; 4 ; 9 ; 12 ; 7]
2	3 à 5	2 (valeur 4)	$T[2] \leftrightarrow T[2]$ (sans effet)	[1 ; 4 ; 9 ; 12 ; 7]
3	4 à 5	5 (valeur 7)	$T[3] \leftrightarrow T[5]$	[1 ; 4 ; 7 ; 12 ; 9]
4	5 à 5	5 (valeur 9)	$T[4] \leftrightarrow T[5]$	[1 ; 4 ; 7 ; 9 ; 12]

Les valeurs en gras forment la partie triée. On vérifie qu'après le passage i , les i premières cases contiennent bien les i plus petits éléments dans l'ordre : c'est l'invariant annoncé plus haut.

2.6 Efficacité du tri par sélection

Combien de travail cet algorithme effectue-t-il ? Comptons les comparaisons. Au passage 1, la boucle intérieure fait $n - 1$ comparaisons ; au passage 2, $n - 2$ comparaisons ; et ainsi de suite jusqu'à 1 comparaison au dernier passage. Le nombre total de comparaisons est donc :

$$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n(n - 1)}{2}$$

Pour $n = 5$, cela donne $\frac{5 \times 4}{2} = 10$ comparaisons, ce que l'on peut vérifier sur la trace : $4 + 3 + 2 + 1 = 10$.

PROPRIÉTÉ Coût du tri par sélection (admise)

Pour un tableau de n éléments, le tri par sélection effectue :

- au plus $n - 1$ échanges (un par passage — c'est très peu, ce qui est un avantage quand les échanges sont coûteux) ;
- exactement $\frac{n(n - 1)}{2}$ comparaisons, c'est-à-dire de l'ordre de $\frac{n^2}{2}$: on dit que sa **complexité** est quadratique.

Conséquence pratique : si la taille du tableau double, le temps de tri est multiplié par environ 4. Le tri par sélection convient donc aux petits tableaux, mais devient lent pour de grandes quantités de données.

LESAVAIS-TU ? Pourquoi « peu d'échanges » peut être un atout

Le tri par sélection est parfois utilisé dans des situations où déplacer les données coûte cher : par exemple pour réorganiser de gros fichiers sur un support lent. Comme il n'effectue qu'au plus $n - 1$ échanges, il minimise le nombre d'écritures, même s'il fait beaucoup de comparaisons (qui, elles, ne modifient rien). En informatique, le « meilleur » algorithme dépend toujours du contexte d'utilisation !

EXERCICE RÉSOLU 2 Exécution d'un tri par sélection

Énoncé. On considère le tableau $T = [15 ; 3 ; 8 ; 3 ; 11]$. Exécuter le tri par sélection sur ce tableau en donnant l'état du tableau après chaque passage, puis donner le nombre total de comparaisons effectuées.

SOLUTION

Solution détaillée. Le tableau contient $n = 5$ éléments : il y aura $n - 1 = 4$ passages.

Passage 1 : partie non triée $[15 ; 3 ; 8 ; 3 ; 11]$. Le minimum est 3 : il apparaît aux indices 2 et 4. L'algorithme retient le premier rencontré, c'est-à-dire l'indice 2 (car le test est strict : $T[j] < T[\text{indiceMin}]$). On échange $T[1]$ et $T[2]$:

$[\underline{3} ; 15 ; 8 ; 3 ; 11]$

Passage 2 : partie non triée $[15 ; 8 ; 3 ; 11]$. Le minimum est 3 à l'indice 4. On échange $T[2]$ et $T[4]$:

$[\underline{3} ; \underline{3} ; 8 ; 15 ; 11]$

Passage 3 : partie non triée $[8 ; 15 ; 11]$. Le minimum est 8 à l'indice 3, déjà en place :

$[\underline{3} ; \underline{3} ; \underline{8} ; 15 ; 11]$

Passage 4 : partie non triée $[15 ; 11]$. Le minimum est 11 à l'indice 5. On échange $T[4]$ et $T[5]$:

$[\underline{3} ; \underline{3} ; \underline{8} ; \underline{11} ; 15]$

Le tableau trié est $[3 ; 3 ; 8 ; 11 ; 15]$.

Nombre de comparaisons : $\frac{n(n-1)}{2} = \frac{5 \times 4}{2} = 10$ comparaisons (soit $4 + 3 + 2 + 1$), et 3 échanges réellement modifiants (le passage 3 n'a rien changé).

EXERCICE 2 À toi de jouer

1. Exécute le tri par sélection sur le tableau [25 ; 17 ; 31 ; 13 ; 2], en rédigeant la trace complète (passage, indiceMin, échange, état du tableau).
2. Réécris l'algorithme du tri par sélection pour trier un tableau par ordre décroissant. Quelle ligne faut-il modifier ?
3. Un tableau contient déjà ses éléments triés par ordre croissant. Le tri par sélection fera-t-il moins de comparaisons que d'habitude ? Justifie.

CORRIGÉ Exercice : tri par sélection

1. $n = 5$, donc 4 passages. Passage 1 : minimum 2 à l'indice 5, échange $T[1] \leftrightarrow T[5]$: [2 ; 17 ; 31 ; 13 ; 25]. Passage 2 : minimum de [17 ; 31 ; 13 ; 25] : 13 à l'indice 4, échange $T[2] \leftrightarrow T[4]$: [2 ; 13 ; 31 ; 17 ; 25]. Passage 3 : minimum de [31 ; 17 ; 25] : 17 à l'indice 4, échange $T[3] \leftrightarrow T[4]$: [2 ; 13 ; 17 ; 31 ; 25]. Passage 4 : minimum de [31 ; 25] : 25 à l'indice 5, échange $T[4] \leftrightarrow T[5]$: [2 ; 13 ; 17 ; 25 ; 31]. Le tableau est trié.
2. Il suffit de remplacer le test Si $T[j] < T[\text{indiceMin}]$ par Si $T[j] > T[\text{indiceMin}]$: on recherche alors le maximum de la partie non triée, que l'on place à gauche. On renommera logiquement la variable `indiceMax`.
3. Non. Le nombre de comparaisons est toujours $\frac{n(n-1)}{2}$, car les deux boucles Pour parcourent toujours les mêmes plages d'indices, que le tableau soit déjà trié ou non. Seuls les échanges deviennent sans effet (on échange une case avec elle-même). C'est une caractéristique du tri par sélection : son coût en comparaisons ne dépend pas de l'ordre initial des données.

En résumé, le tri par sélection est un algorithme simple et rigoureux : $n - 1$ passages, à chaque passage une recherche du minimum dans la partie non triée suivie d'un échange en trois affectations. Sa simplicité le rend facile à exécuter à la main — une compétence exigée au Probatoire — mais son coût quadratique le réserve aux petits tableaux. Dans la section suivante, nous découvrirons un autre tri tout aussi fondamental, le **tri par insertion**, qui adopte une stratégie différente : insérer chaque nouvel élément à sa place dans la partie déjà triée.

3 LE TRI PAR INSERTION

Dans la section précédente, nous avons découvert le **tri par sélection** : à chaque étape, on parcourt la partie non triée du tableau pour y chercher le minimum, puis on l'échange avec le premier élément de cette partie. Cette stratégie fonctionne, mais elle impose de balayer toute la zone non triée à chaque passage, même si le tableau est déjà presque en ordre. Il existe une

autre stratégie, beaucoup plus naturelle, que tu pratiques sans le savoir chaque fois que tu ranges des cartes en main ou que tu classes tes cahiers du plus fin au plus épais : le **tri par insertion**.

3.1 Principe du tri par insertion

Intuition : la main de cartes

Imagine que tu joues aux cartes avec des amis dans un cybercafé de Yaoundé pendant la pause. Tu ramasses tes cartes une par une. À chaque nouvelle carte, tu ne cherches pas la plus petite carte du paquet : tu prends simplement la carte que tu viens de piocher et tu l'insères à sa place parmi les cartes déjà rangées dans ta main, en écartant vers la droite celles qui sont plus grandes. C'est exactement l'idée du tri par insertion.

DÉFINITION Tri par insertion

Le **tri par insertion** est un algorithme de tri qui construit progressivement une partie triée à gauche du tableau. Pour chaque nouvel élément (à partir du deuxième), on le sauvegarde, on décale vers la droite tous les éléments de la partie triée qui lui sont supérieurs, puis on l'insère dans la case ainsi libérée. Après le traitement de l'élément d'indice i , les éléments d'indices 1 à i sont triés entre eux.

PROPRIÉTÉ Invariant du tri par insertion

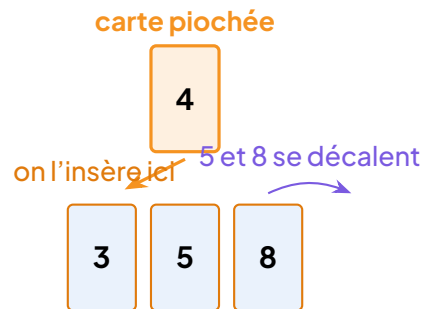
À tout moment de l'exécution, le tableau est divisé en deux zones :

- une partie triée (à gauche), qui grandit d'une case à chaque étape ;
- une partie non triée (à droite), dont on prend les éléments un par un, dans l'ordre.

Au départ, la partie triée contient un seul élément (le premier, qui est forcément « trié » à lui tout seul). À la fin, elle contient tout le tableau.

La différence de philosophie avec le tri par sélection est essentielle :

Critère	Tri par sélection	Tri par insertion
Idee générale	Chercher le minimum de la partie non triée	Insérer chaque élément à sa place dans la partie triée
Zone parcourue	La partie non triée (recherche du min)	La partie triée (recherche de la position)
Action principale	Un échange par étape	Des décalages puis une insertion
Tableau presque trié	Aucun gain : on balaye tout	Très rapide : presque aucun décalage



Analogie de la main de cartes : la carte 4 vient d'être piochée ; on la soulève, on écarte les cartes plus grandes (5 et 8) vers la droite, puis on la glisse entre 3 et 5.

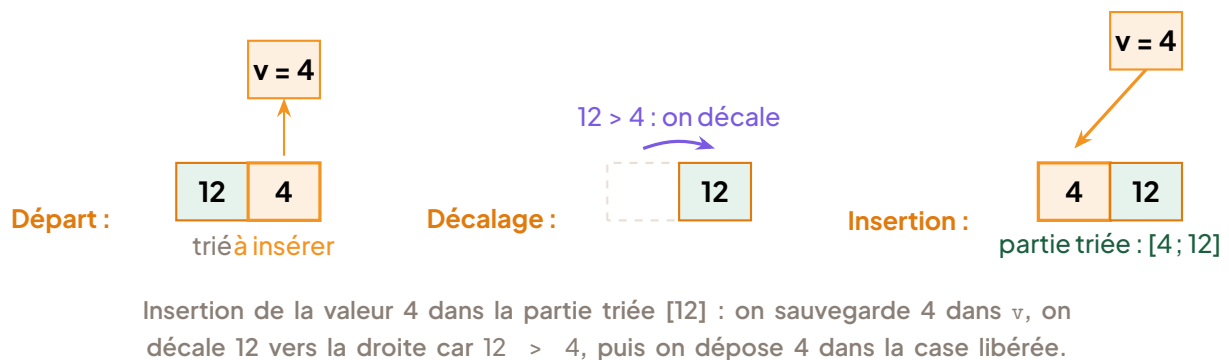
3.2 Le mécanisme du décalage

Le point technique délicat du tri par insertion est le **décalage**. Quand on veut insérer une valeur v dans la partie triée, on ne peut pas simplement « l'écraser » sur une case : on perdrait la valeur qui s'y trouve. On procède donc comme au jeu de taquin :

1. on **sauvegarde** v dans une variable temporaire ;
2. tant que l'élément situé juste à gauche de la case libre est **plus grand que** v , on le **recopie d'une case vers la droite** (la case libre se déplace donc vers la gauche) ;
3. quand on ne peut plus reculer (début du tableau atteint, ou élément inférieur ou égal rencontré), on **dépose** v dans la case libre.

ATTENTION Erreur classique : écraser l'élément à insérer

L'erreur la plus fréquente chez les débutants est d'oublier la sauvegarde. Si tu écris directement $T[j] \leftarrow T[j-1]$ en boucle sans avoir d'abord copié la valeur à insérer dans une variable v , cette valeur est définitivement perdue dès le premier décalage : le tableau contiendra des doublons et le résultat sera faux. Règle d'or : toujours sauvegarder avant de décaler, et ne réinsérer qu'à la toute fin, une fois la boucle de décalage terminée.



3.3 Exécution guidée pas à pas

MÉTHODE Exécuter un tri par insertion à la main

Pour trier un tableau de n nombres à la main :

1. Considère le premier élément comme une partie triée d'une seule case.
2. Pour chaque indice i de 2 à n : entoure (ou recopie à part) la valeur $T[i]$: c'est l'élément à insérer.
3. Compare-la aux éléments de la partie triée, de droite à gauche. Barre et réécris d'une case à droite chaque élément plus grand qu'elle.
4. Dès que tu rencontres un élément inférieur ou égal (ou le début du tableau), écris la valeur sauvegardée dans la case libre.
5. Souligne la nouvelle partie triée (indices 1 à i) et passe à l'élément suivant.

Vérifie à chaque étape que la partie soulignée est bien en ordre croissant : c'est l'invariant qui te protège des erreurs.

EXEMPLE Tri complet de $[12; 4; 9; 1; 7]$ par insertion

Appliquons la méthode au tableau $T = [12; 4; 9; 1; 7]$. La partie triée est indiquée en gras.

État initial : $[12; 4; 9; 1; 7]$ — la partie triée est $[12]$.

Étape 1 — insérer 4 : sauvegarde $v = 4$. Comparaison : $12 > 4$, on décale 12. Début du tableau atteint : on dépose 4.

État : $[4; 12; 9; 1; 7]$ (1 décalage).

Étape 2 — insérer 9 : $v = 9$. Comparaison : $12 > 9$, on décale 12. Puis $4 \leq 9$: on s'arrête et on dépose 9 après le 4.

État : $[4; 9; 12; 1; 7]$ (1 décalage).

Étape 3 — insérer 1 : $v = 1$. $12 > 1$: décalage. $9 > 1$: décalage. $4 > 1$: décalage. Début

atteint : on dépose 1 tout à gauche.

État : [1; 4; 9; 12; 7] (3 décalages).

Étape 4 — insérer 7 : $v = 7$. $12 > 7$: décalage. $9 > 7$: décalage. $4 \leq 7$: on dépose 7 après le 4.

État : [1; 4; 7; 9; 12] (2 décalages).

Le tableau est trié. Total : $1 + 1 + 3 + 2 = 7$ décalages et 4 insertions.

3.4 Algorithme complet du tri par insertion

PROPRIÉTÉ Structure de l'algorithme

Le tri par insertion s'écrit avec deux boucles imbriquées :

- une boucle **Pour** qui fait passer chaque élément, de l'indice 2 à n ;
- une boucle **Tant que** qui effectue les décalages vers la droite tant que l'on n'a pas atteint le début du tableau et que l'élément de gauche est plus grand que la valeur sauvegardée.

Algorithme TriInsertion

Variable

T : Tableau[1..n] de Reel

i, j : Entier

v : Reel

Debut

Pour i De 2 A n Faire

 v ← T[i] // sauvegarde de l'élément à insérer

 j ← i - 1 // on démarre à gauche de la case libre

 Tant que (j >= 1) ET (T[j] > v) Faire

 T[j+1] ← T[j] // décalage vers la droite

 j ← j - 1 // la case libre recule

 Fin Tant que

 T[j+1] ← v // insertion dans la case libérée

Fin Pour

Fin

Lisons cet algorithme ligne par ligne. La variable v joue le rôle de la carte soulevée au-dessus de la main : elle protège la valeur pendant les décalages. La condition double du **Tant que** est cruciale : $j \geq 1$ empêche de sortir du tableau par la gauche, et $T[j] > v$ arrête les décalages dès qu'on a trouvé la bonne position. Remarque enfin que l'insertion $T[j+1] \leftarrow v$ se fait **après** la boucle, jamais dedans.

ATTENTION Pièges de syntaxe et de logique

- Ordre des conditions : écris $(j \geq 1) \text{ ET } (T[j] > v)$ et non l'inverse ; si j vaut 0, éva-

luer $T[j]$ en premier provoquerait un accès hors du tableau.

- Indice d'insertion : après la boucle, la case libre est $j+1$, pas j . Le décalage $T[j+1] \leftarrow T[j]$ a « poussé » la case libre vers la gauche.
- Départ à l'indice 2 : la boucle `Pour` commence à 2, car le premier élément seul est déjà trié. Commencer à 1 serait inutile (et dangereux avec $j \leftarrow i - 1$).

3.5 Cas particuliers et efficacité

PROPRIÉTÉ Comportement selon l'état initial du tableau

- Tableau déjà trié (ex. $[1; 4; 7; 9; 12]$) : pour chaque élément, la condition $T[j] > v$ est fausse dès le premier test : aucun décalage. Le tri ne fait que $n-1$ comparaisons. C'est le meilleur cas.
- Tableau trié en ordre inverse (ex. $[12; 9; 7; 4; 1]$) : chaque élément doit être ramené tout au début, donc tous les décalages possibles ont lieu : $1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2}$ décalages. C'est le pire des cas.

En moyenne, le tri par insertion effectue de l'ordre de $\frac{n^2}{4}$ opérations : comme le tri par sélection, c'est un tri « quadratique », adapté aux petits tableaux (quelques dizaines ou centaines d'éléments).

LESAVAIS-TU ? Le champion des tableaux presque triés

Le tri par insertion possède une qualité rare : il est adaptatif. Plus le tableau est proche de l'ordre final, plus il est rapide. C'est pourquoi les bibliothèques de tri professionnelles (comme celle du langage Python, qui utilise le tri « TimSort ») basculent sur un tri par insertion pour les petits sous-tableaux presque ordonnés. Même les cartes à jouer professionnelles, lors des tournois, sont souvent rangées ainsi par les croupiers : c'est la méthode la plus économique en gestes quand le paquet est déjà presque en ordre !

EXERCICE RÉSOLU 3 Insertion d'un élément dans un tableau presque trié

Énoncé. Le tableau $T = [2; 5; 8; 11; 6]$ contient les tailles (en cm) de cinq élèves rangés presque dans l'ordre. Décris précisément l'exécution du tri par insertion sur ce tableau : nombre d'étapes de la boucle `Pour`, décalages effectués à chaque étape, état final.

SOLUTION

Solution. La partie triée initiale est [2].

Étape 1 — insérer 5 : $v = 5$, $j = 1$. Test : $T[1] = 2 > 5$? Non. Aucun décalage, on repose 5 à sa place. État : [2; 5; 8; 11; 6].

Étape 2 — insérer 8 : $5 > 8$? Non. Aucun décalage. État : [2; 5; 8; 11; 6].

Étape 3 — insérer 11 : $8 > 11$? Non. Aucun décalage. État : [2; 5; 8; 11; 6].

Étape 4 — insérer 6 : $v = 6$, $j = 4$. $T[4] = 11 > 6$: décalage de 11 ($T[5] \leftarrow 11$), $j = 3$. $T[3] = 8 > 6$: décalage de 8 ($T[4] \leftarrow 8$), $j = 2$. $T[2] = 5 > 6$? Non : arrêt. Insertion : $T[3] \leftarrow 6$.

État final : [2; 5; 6; 8; 11]. Le tableau est trié avec seulement 2 décalages au total, alors qu'un tri par sélection aurait balayé toute la partie non triée à chaque étape. Cela illustre parfaitement l'efficacité du tri par insertion sur un tableau presque trié.

EXERCICE 3 À toi de jouer

Exécute à la main le tri par insertion sur le tableau [15 ; 3 ; 10 ; 3 ; 8] en rédigeant, pour chaque étape, la valeur sauvegardée, les décalages effectués et l'état du tableau. Que remarques-tu lors de l'insertion du deuxième 3 (cas des valeurs égales)? L'algorithme donné ci-dessus est-il « stable », c'est-à-dire conserve-t-il l'ordre relatif des éléments égaux?

CORRIGÉ Exercice : tri par insertion de [15 ; 3 ; 10 ; 3 ; 8]

Étape 1 — insérer 3 : $15 > 3$: décalage. Début atteint. État : [3; 15; 10; 3; 8].

Étape 2 — insérer 10 : $15 > 10$: décalage ; $3 \leq 10$: arrêt. État : [3; 10; 15; 3; 8].

Étape 3 — insérer le deuxième 3 : $15 > 3$, $10 > 3$: deux décalages ; puis $T[1] = 3 > 3$? Non, car la condition est stricte ($>$). On s'arrête donc après le premier 3. État : [3; 3; 10; 15; 8]. Le deuxième 3 est placé derrière le premier : l'ordre relatif des éléments égaux est conservé, le tri est stable.

Étape 4 — insérer 8 : $15 > 8$, $10 > 8$: deux décalages ; $3 \leq 8$: arrêt. État final : [3; 3; 8; 10; 15].

La stabilité vient du choix de l'inégalité stricte $T[j] > v$: avec $\geq$, les éléments égaux seraient inversés et le tri perdrait sa stabilité.

À RETENIR L'essentiel du tri par insertion

- Le tri par insertion construit une partie triée à gauche en insérant chaque nouvel élément à sa place, comme on range des cartes en main.

- Trois gestes indispensables : sauvegarder la valeur, décaler vers la droite les éléments plus grands, insérer dans la case libérée.
- L'algorithme combine une boucle `Pour` (indices 2 à n) et une boucle `Tant que` de décalage, avec la double condition $(j \geq 1)$ ET $(T[j] > v)$.
- Coût : quadratique en général (de l'ordre de n^2 opérations), mais quasi instantané sur un tableau presque trié — son grand avantage sur le tri par sélection.
- Avec l'inégalité stricte, le tri par insertion est stable : les éléments égaux gardent leur ordre d'origine.

4 LA RECHERCHE SÉQUENTIELLE

Dans les sections précédentes, nous avons appris à ordonner un tableau grâce au tri par sélection et au tri par insertion. Ordonner, c'est bien ; mais à quoi sert un tableau ordonné (ou même non ordonné) si l'on ne sait pas retrouver une information à l'intérieur ? C'est précisément l'objet des deux dernières sections de cette leçon : les **algorithmes de recherche**. Nous commençons par le plus naturel et le plus universel d'entre eux : la **recherche séquentielle**.

4.1 Situation problème et intuition

Imagine la bibliothèque de ton lycée. Les livres sont posés sur une étagère dans le désordre : un roman de Mongo Beti côtoie un manuel de mathématiques, suivi d'un dictionnaire. Tu cherches le livre « Physique Première ». Quelle est ta seule stratégie possible ? Tu prends le premier livre, tu regardes son titre ; ce n'est pas le bon, tu passes au suivant ; et ainsi de suite, jusqu'à tomber sur le bon livre ... ou jusqu'à épuiser l'étagère et conclure que le livre n'y est pas.

C'est exactement ce que fait un programme lorsqu'il cherche une valeur dans un tableau dont il ne connaît rien de l'organisation : il examine les cases une par une, dans l'ordre, du premier indice au dernier. Cette stratégie, simple et sans aucune hypothèse préalable, porte le nom de **recherche séquentielle** (on dit aussi **recherche linéaire**, car on avance en ligne droite).

DÉFINITION Recherche séquentielle

La **recherche séquentielle** (ou recherche linéaire) est un algorithme de recherche qui consiste à parcourir le tableau du premier élément au dernier, en comparant chaque élément à la valeur cherchée. Le parcours s'arrête dès que :

- l'élément est trouvé : on renvoie sa position (son indice) dans le tableau ;
- la fin du tableau est atteinte sans succès : on conclut que l'élément est absent (on renvoie par convention la valeur -1).

PROPRIÉTÉ Aucune condition sur le tableau

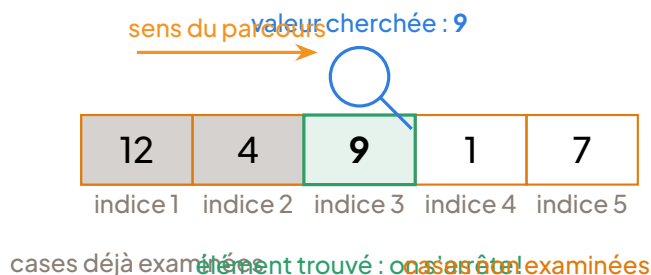
Contrairement à la recherche dichotomique que nous étudierons dans la section suivante, la recherche séquentielle n'exige aucune condition particulière sur le tableau : il n'a pas besoin d'être trié. C'est sa grande force : elle fonctionne sur n'importe quel tableau, dans n'importe quel état. Sa faiblesse, en contrepartie, est son coût : dans le pire des cas, il faut examiner tous les éléments.

4.2 Principe détaillé : trois ingrédients

Pour construire une recherche séquentielle correcte, il faut maîtriser trois ingrédients :

1. **Un indice de parcours i** , initialisé au premier indice du tableau (en algorithmique, nous adoptons la convention où le premier élément est à l'indice 1).
2. **Une variable booléenne `trouve`**, initialisée à Faux, qui mémorise si la valeur a déjà été rencontrée.
3. **Une boucle `Tant que` à double condition** : on continue tant que l'on n'a pas trouvé **et** que l'on n'a pas dépassé la fin du tableau. À chaque tour, on compare la case courante à la valeur cherchée, puis on avance d'une case.

La double condition est essentielle : elle garantit l'arrêt dès que l'élément est trouvé (inutile de continuer à fouiller!) et l'arrêt en fin de tableau si l'élément est absent.



Recherche de la valeur 9 dans le tableau [12; 4; 9; 1; 7] : les cases 1 et 2 ont été examinées (grisées), la loupe s'arrête sur la case 3 qui contient la valeur cherchée. Les cases 4 et 5 ne seront jamais consultées.

4.3 Méthode : écrire une recherche séquentielle pas à pas

MÉTHODE Construire l'algorithme de recherche séquentielle

Pour écrire correctement une recherche séquentielle, suis toujours ces cinq étapes :

1. Initialisation : poser $i \leftarrow 1$ (premier indice) et `trouve` $\leftarrow$ Faux.
2. Condition d'arrêt : écrire la boucle Tant que ($i \leq n$) ET (`trouve` = Faux). Les deux conditions sont reliées par un ET logique : la boucle s'arrête dès que l'une des deux devient fausse.
3. Comparaison : dans le corps de la boucle, tester Si $T[i] = \text{val}$; si oui, mémoriser la position et passer `trouve` à Vrai.
4. Avancement : incrémenter l'indice ($i \leftarrow i + 1$) dans tous les cas, pour progresser vers la fin du tableau.
5. Conclusion : après la boucle, si `trouve` vaut Vrai, renvoyer la position mémorisée; si-non renvoyer -1 (convention pour « absent »).

EXEMPLE Algorithme complet de recherche séquentielle

Voici l'algorithme, en langage algorithmique classique, qui recherche la valeur `val` dans un tableau `T` de `n` entiers et renvoie sa position (ou -1 si elle est absente) :

```
Algorithme RechercheSequentielle
Variable
  T : Tableau[1..100] d'Entier
  n, i, position : Entier
  val : Entier
  trouve : Booleen
Debut
  // Saisie de la valeur cherchée
  Ecrire("Valeur à rechercher : ")
  Lire(val)

  // 1. Initialisation
  i  $\leftarrow$  1
  trouve  $\leftarrow$  Faux
  position  $\leftarrow$  -1

  // 2. Boucle à double condition d'arrêt
  Tant que (i  $\leq$  n) ET (trouve = Faux) Faire
    // 3. Comparaison de la case courante
    Si  $T[i] = \text{val}$  Alors
      trouve  $\leftarrow$  Vrai
      position  $\leftarrow$  i
    FinSi
    // 4. Avancement dans tous les cas
    i  $\leftarrow$  i + 1
  FinTant que

  // 5. Conclusion
  Si trouve = Vrai Alors
    Ecrire("Élément trouvé à la position ", position)
  Sinon
    Ecrire("Élément absent du tableau")
```

FinSi
Fin

Remarque que la variable `position` est initialisée à `-1` : si la boucle se termine sans que l'élément ait été trouvé, `position` conserve cette valeur conventionnelle qui signifie « absent ». On peut donc se passer du booléen dans la conclusion en testant simplement `position <> -1`.

4.4 Exécution à la main sur un exemple

Exécutons pas à pas cet algorithme sur le tableau $T = [12 ; 4 ; 9 ; 1 ; 7]$ (donc $n = 5$), d'abord pour une valeur présente, puis pour une valeur absente.

Premier cas : recherche de la valeur 9 (présente).

Tour	i	$T[i]$	Comparaison $T[i] = 9$?	État après le tour
1	1	12	Non	<code>trouve = Faux</code> , $i \leftarrow 2$
2	2	4	Non	<code>trouve = Faux</code> , $i \leftarrow 3$
3	3	9	Oui	<code>trouve = Vrai</code> , <code>position</code> $\leftarrow 3$, $i \leftarrow 4$

Au début du quatrième test de la boucle, la condition `trouve = Faux` est devenue fausse : la boucle s'arrête immédiatement. L'algorithme affiche « Élément trouvé à la position 3 ». Il a fallu **3 comparaisons** seulement : les cases 4 et 5 n'ont jamais été examinées.

Deuxième cas : recherche de la valeur 20 (absente).

Tour	i	$T[i]$	Comparaison $T[i] = 20$?	État après le tour
1	1	12	Non	$i \leftarrow 2$
2	2	4	Non	$i \leftarrow 3$
3	3	9	Non	$i \leftarrow 4$
4	4	1	Non	$i \leftarrow 5$
5	5	7	Non	$i \leftarrow 6$

Après le cinquième tour, i vaut 6 : la condition $i \leq n$ (c'est-à-dire $6 \leq 5$) devient fausse, la boucle s'arrête. Comme `trouve` vaut toujours `Faux`, l'algorithme affiche « Élément absent du tableau ». Il a fallu **5 comparaisons**, c'est-à-dire le parcours complet du tableau : c'est le pire des cas.

À RETENIR Coût de la recherche séquentielle

Pour un tableau de n éléments, le nombre de comparaisons effectuées par la recherche séquentielle est :

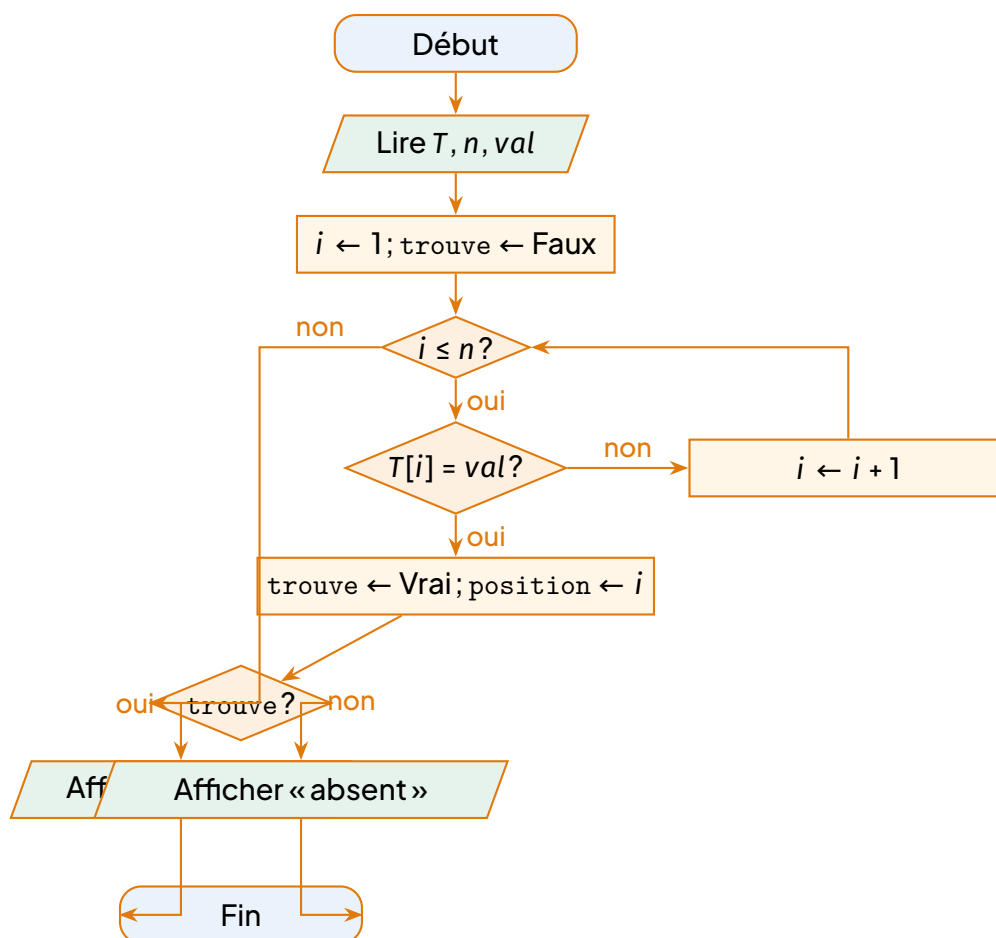
- meilleur cas : 1 comparaison (l'élément cherché est en première position) ;

- cas moyen : environ $\frac{n}{2}$ comparaisons si l'élément est présent ;
- pire des cas : n comparaisons (élément absent ou en dernière position).

On dit que le coût est linéaire en n : si la taille du tableau double, le temps de recherche double aussi.

4.5 Organigramme de la recherche séquentielle

L'organigramme ci-dessous résume la logique de l'algorithme. Observe bien les deux portes de sortie de la boucle : la sortie « trouvé » (par le haut du losange de comparaison) et la sortie « fin du tableau » (par la condition $i \leq n$).



Organigramme simplifié de la recherche séquentielle : la boucle est formée par les deux losanges et le retour de l'incréméntation vers le test $i \leq n$.

4.6 Les pièges classiques à éviter

ATTENTION Erreurs fréquentes dans la recherche séquentielle

Trois bugs reviennent sans cesse dans les copies et les programmes des élèves :

1. Boucle infinie : si tu oublies l'instruction $i \leftarrow i + 1$ dans le corps de la boucle, l'indice ne progresse jamais, la condition $i \leq n$ reste vraie éternellement (tant que l'élément n'est pas trouvé) et le programme plante en tournant indéfiniment. Vérifie toujours que l'incrément est présente et placée à l'intérieur de la boucle, hors du Si.
2. Dépassement d'indice : si la condition sur `trouve` est mal placée (par exemple Tant que (`trouve = Faux`) ET ($i \leq n$) avec un langage qui évalue d'abord $T[i]$), ou si la boucle est mal bornée ($i < n$ au lieu de $i \leq n$, ou l'inverse), on risque soit de lire la case $T[n+1]$ qui n'existe pas (erreur d'exécution), soit d'oublier d'examiner la dernière case. La condition correcte est Tant que ($i \leq n$) ET (`trouve = Faux`).
3. Arrêt oublié : certains élèves parcourent tout le tableau même après avoir trouvé l'élément. Ce n'est pas faux si l'on veut la dernière occurrence, mais c'est inefficace et, surtout, si l'on écrase `position` à chaque occurrence, on ne renvoie plus la première position. Dès que l'élément est trouvé, il faut sortir de la boucle.

4.7 Exercice résolu

EXERCICE RÉSOLU 4 Recherche dans les notes d'une classe

Le tableau $N = [14 ; 8 ; 15 ; 10 ; 8 ; 17]$ contient les notes de six élèves de Première TI. On exécute l'algorithme de recherche séquentielle pour chercher la valeur 8.

1. Combien de comparaisons sont effectuées ? Quelle position est renvoyée ?
2. Même question pour la valeur 11.

SOLUTION

1. Recherche de la valeur 8. On parcourt le tableau :

- Tour 1 : $N[1] = 14 \neq 8$; on avance, $i \leftarrow 2$.
- Tour 2 : $N[2] = 8 = 8$: trouvé ! On mémorise `position` $\leftarrow 2$ et `trouve` devient Vrai, ce qui stoppe la boucle.

L'algorithme effectue 2 comparaisons et renvoie la position 2. Remarque : la valeur 8 apparaît aussi en position 5, mais la recherche séquentielle renvoie la première occurrence, car elle s'arrête dès le premier succès.

2. Recherche de la valeur 11. On compare successivement 14, 8, 15, 10, 8, 17 à 11 : aucune case ne convient. Après le sixième tour, i vaut $7 > 6 = n$: la boucle s'arrête sur la condition de fin de tableau. L'algorithme effectue donc 6 comparaisons (n comparaisons : pire des cas) et renvoie -1 : la valeur 11 est absente du tableau.

LESAVAIS-TU ? Et dans la vraie vie ?

Quand ton téléphone cherche un contact dans une liste non triée, ou quand le logiciel de la cantine vérifie si un matricule figure dans le fichier des élèves inscrits, c'est très souvent une recherche séquentielle qui est exécutée en coulisses — du moins tant que les données ne sont pas triées. Pour de petites listes (quelques centaines d'éléments), elle est largement assez rapide : un ordinateur moderne effectue des milliards de comparaisons par seconde. C'est seulement sur de très gros volumes (les millions de clients de CAMTEL ou de MTN, par exemple) que son coût linéaire devient rédhibitoire et qu'il faut trier les données pour utiliser la recherche dichotomique, objet de la prochaine section.

EXERCICE 4 À toi de jouer

On considère le tableau $P = [35 ; 22 ; 48 ; 10 ; 22 ; 5]$ représentant les prix (en centaines de FCFA) d'articles d'une boutique.

1. Exécute à la main la recherche séquentielle de la valeur 48 : dresse le tableau des tours (indice, valeur comparée, résultat) et donne la position renvoyée.
2. Même question pour la valeur 100.
3. Modifie l'algorithme pour qu'il compte et affiche le nombre de comparaisons effectuées.

CORRIGÉ Exercice : recherche dans les prix

1. Valeur 48 : tour 1, $P[1] = 35 \neq 48$; tour 2, $P[2] = 22 \neq 48$; tour 3, $P[3] = 48 = 48$: trouvé. Position renvoyée : 3, en 3 comparaisons.
2. Valeur 100 : les six cases sont examinées sans succès (35, 22, 48, 10, 22, 5). Après le sixième tour, $i = 7 > 6$: arrêt. Résultat : -1, valeur absente, en 6 comparaisons (pire des cas).
3. Il suffit d'ajouter une variable `nbComp` initialisée à 0 et incrémentée juste avant chaque comparaison :

```
nbComp <- 0
Tant que (i <= n) ET (trouve = Faux) Faire
  nbComp <- nbComp + 1 // une comparaison va être faite
  Si T[i] = val Alors
    trouve <- Vrai
    position <- i
  FinSi
  i <- i + 1
FinTant que
Ecrire("Nombre de comparaisons : ", nbComp)
```

En résumé, la recherche séquentielle est l'algorithme de recherche universel : simple à écrire, correct sur tout tableau, trié ou non, au prix d'un coût pouvant atteindre n comparaisons. Mais si le tableau est trié — et nous savons désormais le trier ! — peut-on faire mieux que d'examiner les cases une par une ? C'est toute l'idée de la **recherche dichotomique**, que nous étudions dans la section suivante.

5 LA RECHERCHE DICHOTOMIQUE

Dans la section précédente, nous avons étudié la **recherche séquentielle** : on parcourt le tableau élément par élément, du premier au dernier, jusqu'à trouver la valeur cherchée. Cette méthode est simple et fonctionne sur n'importe quel tableau, mais elle a un défaut majeur : si l'élément cherché est le dernier, ou s'il est absent, on effectue n comparaisons pour un tableau de n éléments. Pour un tableau de 1 000 000 d'élèves dans la base de données d'un ministère, cela peut faire jusqu'à 1 000 000 comparaisons !

Imagine maintenant que tu cherches le mot « harmonie » dans un dictionnaire. Tu ne lis certainement pas toutes les pages depuis le début ! Tu ouvres le dictionnaire au milieu, tu regardes où tu tombes : si tu es à la lettre « M », tu sais que « harmonie » est avant, donc tu ignores toute la seconde moitié. Tu recommences avec la moitié restante, et ainsi de suite. En quelques ouvertures seulement, tu trouves ton mot parmi des milliers de pages. C'est exactement le principe de la **recherche dichotomique** (du grec *dichotomia* : « coupure en deux »).

5.1 Condition indispensable : un tableau trié

ATTENTION La recherche dichotomique exige un tableau trié

La recherche dichotomique n'est correcte que si le tableau est **trié** (par exemple en ordre croissant). Si le tableau n'est pas trié, l'algorithme peut éliminer la moitié qui contient pourtant l'élément cherché, et conclure à tort qu'il est absent. Avant toute recherche dichotomique, il faut donc vérifier que le tableau est trié, ou le trier au préalable (avec le tri par sélection ou le tri par insertion, vus dans les sections précédentes).

Pourquoi cette condition ? Parce que tout le raisonnement repose sur le fait suivant : si le tableau est trié en ordre croissant et que l'élément du milieu est plus petit que la valeur cherchée, alors **tous** les éléments situés à gauche du milieu sont aussi plus petits que la valeur cherchée : inutile de les examiner. On élimine d'un coup la moitié du tableau, ce qui est impossible si les valeurs sont rangées en désordre.

5.2 Principe et variables de gestion

DÉFINITION Recherche dichotomique

La **recherche dichotomique** est un algorithme de recherche d'un élément dans un ta-

bleau trié. À chaque étape, on compare la valeur cherchée à l'élément situé au milieu de la zone de recherche, puis on ne conserve que la moitié de la zone susceptible de contenir la valeur. On répète jusqu'à trouver l'élément ou jusqu'à ce que la zone de recherche soit vide.

Pour délimiter la zone de recherche courante, on utilise trois variables entières :

- **debut** : indice de la borne inférieure de la zone de recherche ;
- **fin** : indice de la borne supérieure de la zone de recherche ;
- **m** : indice de l'élément du milieu, calculé par $m = (\text{debut} + \text{fin}) \text{ div } 2$ (division entière).

Au départ, la zone de recherche est le tableau entier : $\text{debut} \leftarrow 1$ et $\text{fin} \leftarrow n$ (pour un tableau de n éléments indicé de 1 à n).

PROPRIÉTÉ Les trois cas possibles à chaque étape

Soit val la valeur cherchée dans un tableau T trié en ordre croissant, et m l'indice du milieu de la zone courante. Trois cas se présentent :

1. $T[m] = val$: l'élément est trouvé, la recherche s'arrête ;
2. $T[m] < val$: la valeur cherchée, si elle existe, se trouve à droite du milieu ; on pose $\text{debut} \leftarrow m + 1$;
3. $T[m] > val$: la valeur cherchée, si elle existe, se trouve à gauche du milieu ; on pose $\text{fin} \leftarrow m - 1$.

ATTENTION Ne pas se tromper de côté

C'est l'erreur classique des élèves : si $T[m] < val$, c'est bien la partie droite qu'on conserve (les grandes valeurs sont à droite dans un tableau croissant), donc on déplace debut . Si $T[m] > val$, on conserve la partie gauche, donc on déplace fin . Retiens l'image du dictionnaire : « trop petit $\Rightarrow$ j'avance vers la fin ; trop grand $\Rightarrow$ je recule vers le début ». Autre piège : il faut écrire $m + 1$ et $m - 1$ (et non m), sinon on risque de boucler indéfiniment en réexaminant toujours le même élément du milieu.

La **condition d'arrêt** est double : soit l'élément est trouvé ($T[m] = val$), soit la zone de recherche devient vide, ce qui se traduit par $\text{debut} > \text{fin}$: l'élément est alors **absent** du tableau.

5.3 Exécution guidée pas à pas

MÉTHODE Exécuter une dichotomie « à la main »

Pour exécuter proprement une recherche dichotomique sur le papier (compétence exigée à l'examen), procède ainsi :

1. Écris le tableau trié avec les indices au-dessus des valeurs ;
2. Tiens à jour un tableau de suivi à 4 colonnes : étape, debut, fin, m ;
3. À chaque étape, calcule $m = (debut + fin) \div 2$, compare $T[m]$ à la valeur cherchée, puis mets à jour *debut* ou *fin* selon le cas ;
4. Arrête-toi dès que $T[m] = val$ (trouvé) ou dès que $debut > fin$ (absent).

EXEMPLE Recherche de 9 dans [1 ; 4 ; 7 ; 9 ; 12]

Le tableau $T = [1 ; 4 ; 7 ; 9 ; 12]$ est trié en ordre croissant, avec les indices 1 à 5. On cherche $val = 9$.

Étape 1 : $debut = 1$, $fin = 5$, donc $m = (1 + 5) \div 2 = 3$. Or $T[3] = 7 < 9$: on cherche à droite, $debut \leftarrow 4$.

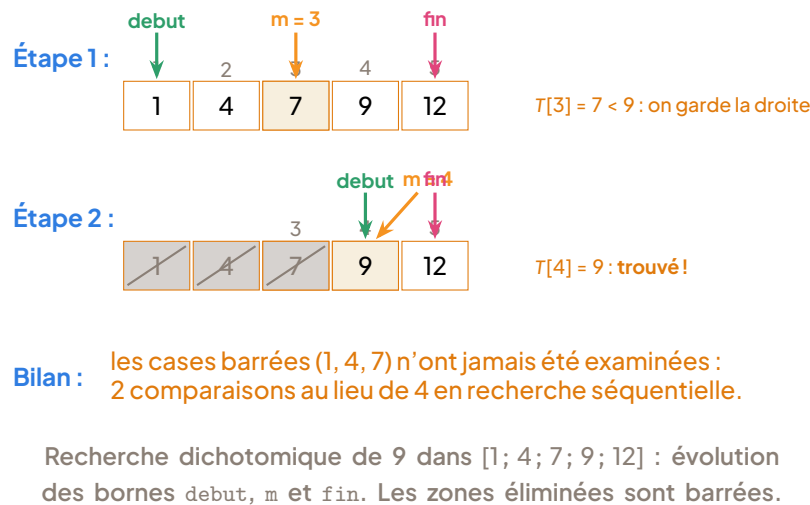
Étape 2 : $debut = 4$, $fin = 5$, donc $m = (4 + 5) \div 2 = 4$. Or $T[4] = 9 = val$: trouvé à l'indice 4 ! Il n'a fallu que 2 comparaisons, contre 4 avec une recherche séquentielle.

EXEMPLE Recherche d'un élément absent : 8 dans [1 ; 4 ; 7 ; 9 ; 12]

Étape 1 : $debut = 1$, $fin = 5$, $m = 3$, $T[3] = 7 < 8$: on va à droite, $debut \leftarrow 4$.

Étape 2 : $debut = 4$, $fin = 5$, $m = 4$, $T[4] = 9 > 8$: on va à gauche, $fin \leftarrow 3$.

Étape 3 : $debut = 4 > fin = 3$: la zone est vide, l'élément 8 est absent. La recherche s'arrête après 2 comparaisons.



5.4 Algorithme complet de recherche dichotomique

Nous pouvons maintenant rassembler toutes les idées précédentes dans un algorithme complet, écrit en langage algorithmique classique. On suppose le tableau T de n éléments **trié en ordre croissant**.

Algorithme RechercheDichotomique

Variable

T : Tableau[1..n] de Reel

n : Entier

val : Reel

debut, fin, m : Entier

trouve : Booléen

Debut

// Saisie de la valeur recherchée

Ecrire("Entrez la valeur à rechercher : ")

Lire(val)

// Initialisation : la zone de recherche est le tableau entier

debut $\leftarrow 1$

fin $\leftarrow n$

trouve $\leftarrow$ Faux

// Tant que la zone n'est pas vide et qu'on n'a pas trouvé

Tant que (debut $\leq$ fin) ET (trouve = Faux) Faire

$m \leftarrow (\text{debut} + \text{fin}) \text{ div } 2$ // indice du milieu

Si $T[m] = val$ Alors

trouve $\leftarrow$ Vrai // cas 1 : élément trouvé

Sinon

Si $T[m] < val$ Alors

debut $\leftarrow m + 1$ // cas 2 : chercher à droite

Sinon

fin $\leftarrow m - 1$ // cas 3 : chercher à gauche

FinSi

FinSi

FinTantQue

// Affichage du résultat

```

Si trouve = Vrai Alors
    Ecrire("Element trouve a l'indice ", m)
Sinon
    Ecrire("Element absent du tableau")
FinSi
Fin

```

Remarque la structure de la boucle : la condition `debut <= fin` garantit l'arrêt lorsque l'élément est absent (la zone devient vide), et le booléen `trouve` permet de sortir dès que l'élément est trouvé, sans attendre que la zone se vide.

EXERCICE RÉSOLU 5 Écrire et exécuter une recherche dichotomique

Énoncé. Une bibliothèque scolaire de Yaoundé gère les codes de ses livres dans un tableau trié : $T = [102; 205; 310; 418; 523; 630; 745]$. 1) Exécute à la main la recherche dichotomique du code 523, en remplissant le tableau de suivi (étape, debut, fin, m). 2) Combien de comparaisons auraient été nécessaires au maximum avec une recherche séquentielle ?

SOLUTION

Solution. 1) Le tableau a $n = 7$ éléments, triés en ordre croissant : la dichotomie est licite.

Étape	debut	fin	m	Comparaison et décision
1	1	7	4	$T[4] = 418 < 523$: droite, $debut \leftarrow 5$
2	5	7	6	$T[6] = 630 > 523$: gauche, $fin \leftarrow 5$
3	5	5	5	$T[5] = 523 = val$: trouvé à l'indice 5

Le code 523 est trouvé en 3 comparaisons.

2) En recherche séquentielle, 523 est le 5^e élément : il aurait fallu 5 comparaisons, et jusqu'à 7 dans le pire des cas (élément absent ou en dernière position). La dichotomie est donc ici plus rapide, et l'écart grandit très vite avec la taille du tableau.

5.5 Efficacité de la recherche dichotomique

Pourquoi cet algorithme est-il si rapide ? Parce qu'à chaque étape, la zone de recherche est divisée par 2. En partant de n éléments, il reste après chaque comparaison environ $n/2$, puis $n/4$, puis $n/8$ éléments à examiner, jusqu'à n'en avoir plus qu'un seul.

PROPRIÉTÉ Nombre maximal d'étapes (admise)

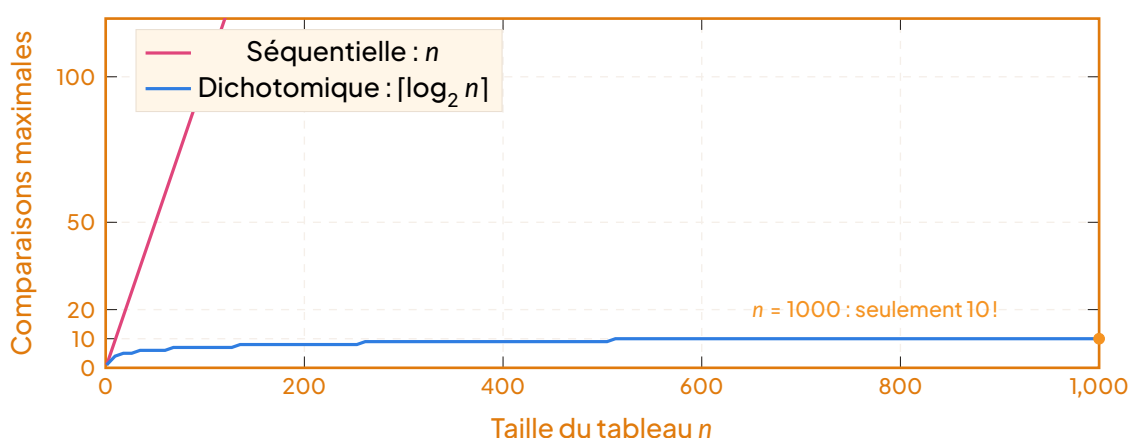
Pour un tableau trié de n éléments, le nombre maximal de comparaisons effectuées par la recherche dichotomique est le plus petit entier k tel que :

$$2^k \geq n$$

On dit que le nombre d'étapes croît comme le logarithme de n (notion admise à ce niveau).

Vérifions sur des exemples concrets :

- $n = 5 : 2^3 = 8 \geq 5$, donc au plus **3** comparaisons (contre 5 en séquentiel);
- $n = 100 : 2^7 = 128 \geq 100$, donc au plus **7** comparaisons (contre 100);
- $n = 1000 : 2^{10} = 1024 \geq 1000$, donc au plus **10** comparaisons (contre 1000);
- $n = 1\,000\,000 : 2^{20} = 1\,048\,576 \geq 10^6$, donc au plus **20** comparaisons!



Comparaison du nombre maximal de comparaisons : la recherche séquentielle croît comme n (droite), la recherche dichotomique comme $\log_2 n$ (courbe presque plate).

Le graphique parle de lui-même : la droite de la recherche séquentielle monte sans frein, tandis que la courbe logarithmique de la dichotomie reste écrasée près de l'axe horizontal. C'est pourquoi tous les moteurs de recherche et toutes les bases de données du monde utilisent des variantes de cette idée.

LESAVAIS-TU ?

Le logarithme, un outil de champion

Le logarithme en base 2, $\log_2 n$, répond à la question : « combien de fois peut-on diviser n par 2 avant d'arriver à 1 ? ». C'est l'une des croissances les plus lentes des mathématiques : doubler la taille des données n'ajoute qu'une seule étape de travail. Passer de 1 000 à 2 000 élèves dans le fichier d'un lycée ne coûte qu'une comparaison supplémentaire à la recherche dichotomique. Cette idée de « diviser pour régner » (divide and conquer) est l'une des stratégies les plus puissantes de toute l'informatique : tu la retrouveras dans de nombreux algorithmes avancés.

À RETENIR L'essentiel de la recherche dichotomique

- La recherche dichotomique s'applique uniquement à un tableau trié; sinon, ses conclusions sont fausses.
- On maintient deux bornes, *debut* et *fin*, et on examine le milieu $m = (debut + fin) \div 2$.
- Trois cas : $T[m] = val$ (trouvé); $T[m] < val$ (on va à droite : $debut \leftarrow m + 1$); $T[m] > val$ (on va à gauche : $fin \leftarrow m - 1$).
- On s'arrête quand l'élément est trouvé ou quand $debut > fin$ (élément absent).
- Efficacité remarquable : au plus k comparaisons avec $2^k \geq n$; pour 1000 éléments, 10 comparaisons suffisent.

EXERCICE 5 À toi de jouer

On considère le tableau trié $T = [3; 8; 15; 21; 34; 47; 52; 68; 90]$.

1. Exécute à la main la recherche dichotomique de la valeur 47, en remplissant le tableau de suivi (étape, *debut*, *fin*, *m*).
2. Exécute la recherche de la valeur 50. Que conclus-tu et après combien d'étapes?
3. Quel est le nombre maximal de comparaisons pour ce tableau de 9 éléments? Vérifie avec la propriété $2^k \geq n$.

CORRIGÉ Exercice : recherche dichotomique

- 1) Recherche de 47 : étape 1 : $debut = 1, fin = 9, m = 5, T[5] = 34 < 47$, donc $debut \leftarrow 6$; étape 2 : $debut = 6, fin = 9, m = 7, T[7] = 52 > 47$, donc $fin \leftarrow 6$; étape 3 : $debut = 6, fin = 6, m = 6, T[6] = 47$: trouvé à l'indice 6 en 3 comparaisons.
- 2) Recherche de 50 : étape 1 : $m = 5, T[5] = 34 < 50$, $debut \leftarrow 6$; étape 2 : $m = 7, T[7] = 52 > 50$, $fin \leftarrow 6$; étape 3 : $m = 6, T[6] = 47 < 50$, $debut \leftarrow 7$; étape 4 : $debut = 7 > fin = 6$: la zone est vide, 50 est absent après 3 comparaisons.
- 3) On cherche le plus petit k tel que $2^k \geq 9$: $2^3 = 8 < 9$ et $2^4 = 16 \geq 9$, donc au maximum 4 comparaisons.

6**SYNTHÈSE : CHOISIR ET ÉCRIRE LE BON ALGORITHME**

Après avoir détaillé le fonctionnement de la recherche dichotomique, nous disposons maintenant de toute la boîte à outils algorithmique prévue au programme de Première TI pour mani-

puler des tableaux : deux tris (sélection, insertion) et deux recherches (séquentielle, dichotomique). La compétence visée ici n'est plus seulement exécuter ou écrire un algorithme donné, mais **choisir** le bon outil face à un problème concret, **justifier** ce choix avec rigueur (condition d'application, coût estimé) et **produire** un code propre, sans erreurs, prêt pour l'épreuve du Probatoire.

6.1 Tableaux comparatifs de synthèse

Pour décider rapidement, rien ne vaut une vue d'ensemble structurée. Le tableau ci-dessous résume les quatre algorithmes vus dans ce module selon leur principe, leur condition d'application et leur coût en nombre de comparaisons (ordre de grandeur pour un tableau de taille n).

Algorithme	Principe clé	Condition d'application	Coût comparaisons (pire cas)
Tri par sélection	Chercher le min dans la partie non triée	Aucune (tableau quelconque)	$\approx \frac{n^2}{2}$ (quadratique)
Tri par insertion	Insérer chaque élément à sa place dans la partie triée	Aucune (tableau quelconque) Idéal si quasi trié	$\approx \frac{n^2}{4}$ (moyen) $\approx \frac{n^2}{2}$ (pire)
Recherche séquentielle	Parcourir le tableau du début à la fin	Aucune (trié ou non)	n (linéaire)
Recherche dichotomique	Comparer au milieu, jeter la moitié inutile, recommencer	OBLIGATOIRE : tableau trié	$\lfloor \log_2 n \rfloor + 1$ (logarithmique)

Tableau de synthèse des algorithmes de tri et de recherche (Première TI).

À RETENIR Règle de choix immédiate

- Tableau non trié, une seule recherche → **Recherche séquentielle** (coût n , pas de tri préalable).
- Tableau non trié, recherches multiples → **Trier une fois** (sélection ou insertion) puis **Recherche dichotomique** pour chaque requête.
- Tableau déjà trié → **Recherche dichotomique** systématiquement (coût $\log_2 n$).
- Tri simple, petit n (< 50) ou quasi trié → **Tri par insertion** (peu d'échanges, code court).
- Tri simple, n moyen, pas quasi trié → **Tri par sélection** (nombre d'échanges minimal : $n - 1$ max).

6.2 Stratégie complète : trier pour mieux chercher

C'est le scénario classique d'une application de gestion (notes d'élèves, stock de boutique, annuaire d'un cybercafé à Yaoundé) : on reçoit des données en vrac, on doit pouvoir y chercher vite.

MÉTHODE Stratégie « Trier une fois, chercher souvent »

1. Analyser le besoin : Combien d'éléments (n) ? Combien de recherches prévues (q) ? Le tableau est-il déjà trié ?
2. Si $q = 1$ (une seule recherche) : Appliquer directement la recherche séquentielle. Coût total $\approx n$. Inutile de payer le tri (n^2).
3. Si $q > 1$ (plusieurs recherches) :
 - (a) Choisir un tri (insertion si quasi trié, sinon sélection).
 - (b) Exécuter le tri une seule fois. Coût $\approx n^2$.
 - (c) Pour chaque recherche, exécuter la dichotomie. Coût par recherche $\approx \log_2 n$.
4. Coût global estimé : $C_{total} \approx n^2 + q \times \log_2 n$.
5. Seuil de rentabilité : Le tri + dichotomie devient rentable dès que $n^2 + q \log_2 n < q \times n$, c'est-à-dire très vite dès que $q \geq 2$ pour $n > 10$.



↓ Tri (Sélection/Insertion)

Tableau initial (non trié)



Tableau trié (ordre croissant)

↓ Recherche dichotomique (ex : chercher 15)

TROUVÉ à l'indice 6

Résultat : position trouvée en $\approx \log_2 8 = 3$ étapes

Stratégie en deux temps : trier le tableau désordonné, puis appliquer la dichotomie.

6.3 Relecture critique : repérer les erreurs classiques (Pièges du Bac)

Au Probatoire, on vous donne souvent un algorithme « presque juste » à corriger, ou on vous demande d'écrire le vôtre. Voici les **quatre erreurs fatales** qui reviennent chaque année sur les copies.

ATTENTION Les 4 pièges à vérifier systématiquement

1. Bornes de boucle (Off-by-one) :

- Tri sélection : boucle extérieure i de 0 à $N-2$ (ou 1 à $N-1$). Boucle intérieure j de $i+1$ à $N-1$. Pas $j \leq N$.
- Tri insertion : boucle extérieure i de 1 à $N-1$. Boucle intérieure TantQue $j \geq 0$ ET $T[j] > cle$. Oublier $j \geq 0 \rightarrow$ débordement indice -1.
- Dichotomie : initialisation $deb \leftarrow 0, fin \leftarrow N-1$. Condition TantQue $deb \leq fin$. Milieu $mil \leftarrow (deb+fin)/2$ (division entière).

2. Échange incorrect (perte de valeur) :

```
// ERREUR CLASSIQUE
T[i] <- T[j]
T[j] <- T[i] // T[i] a déjà été écrasé ! Perte de l'ancienne valeur.

// CORRECT : variable temporaire
temp <- T[i]
T[i] <- T[j]
T[j] <- temp
```

3. Condition d'arrêt de la dichotomie :

- Si $T[mil] < x$: $deb \leftarrow mil + 1$ (on exclut le milieu).
- Si $T[mil] > x$: $fin \leftarrow mil - 1$ (on exclut le milieu).
- Si $T[mil] = x$: trouvé, on sort (retourner mil ou Vrai).

Ne pas mettre $+1/-1 \rightarrow$ boucle infinie si l'élément n'est pas au milieu.

4. Tableau non trié pour la dichotomie : Écrire un algorithme de dichotomie sans vérifier/préciser que le tableau est trié = 0 point à la partie « justification/condition » au Bac. Toujours écrire en commentaire : // Précondition : T trié dans l'ordre croissant.

EXERCICE RÉSOLU 6 Correction d'un algorithme buggé

Énoncé : L'algorithme ci-dessous prétend réaliser un tri par insertion sur un tableau $T[0..N-1]$. Il contient trois erreurs. Les identifier et proposer la version corrigée.

```
Algorithme TriInsertionBugge
Variables : T : Tableau[0..N-1] d'entiers; i, j, cle : Entiers
Début
    Pour i <- 1 à N faire // Erreur 1
        cle <- T[i]
        j <- i // Erreur 2
        TantQue (j > 0) ET (T[j] > cle) faire // Erreur 3
```

```

        T[j+1] <- T[j]
        j <- j - 1
    FinTantQue
    T[j] <- cle           // Erreur 4 (conséquence de 2)
FinPour
Fin

```

SOLUTION

Solution détaillée :

1. Erreur 1 (Borne extérieure) : Pour $i \leftarrow 1$ à N faire. L'indice maximal valide est $N - 1$.
Correction : Pour $i \leftarrow 1$ à $N-1$ faire.
2. Erreur 2 (Initialisation j) : $j \leftarrow i$. On doit comparer `cle` avec l'élément précédent ($i-1$).
Correction : $j \leftarrow i - 1$.
3. Erreur 3 (Condition TantQue) : $T[j] > cle$ accède à $T[i]$ au premier tour (si $j = i$), mais surtout si j devient -1 , on teste $T[-1] \rightarrow$ plantage. Il faut tester $j \geq 0$ avant d'accéder au tableau. Correction : TantQue ($j \geq 0$) ET ($T[j] > cle$) faire. (L'ordre du ET compte en évaluation paresseuse, mais en pseudo-code on écrit la garde d'indice en premier).
4. Erreur 4 (Insertion finale) : Comme j a été décrémenté une fois de trop à la fin du TantQue, l'insertion se fait à $T[j+1]$. Correction : $T[j+1] \leftarrow cle$.

Algorithme corrigé :

```

Algorithme TriInsertionCorrige
Variables : T : Tableau[0..N-1] d'entiers; i, j, cle : Entiers
Début
    Pour i <- 1 à N-1 faire
        cle <- T[i]
        j <- i - 1
        TantQue (j >= 0) ET (T[j] > cle) faire
            T[j+1] <- T[j]
            j <- j - 1
        FinTantQue
        T[j+1] <- cle
    FinPour
Fin

```

6.4 Notion intuitive de complexité : pourquoi la dichotomie est « plus rapide »

Le programme de Première TI n'exige pas de calcul formel de complexité algorithmique (notation $O(\dots)$), mais le jury attend une **intuition quantitative** : « plus n grandit, plus l'écart se creuse ».

EXEMPLE Exemple chiffré : 1024 élèves dans un lycée de Douala

Supposons qu'on cherche la note d'un élève précis dans la liste de la classe de Terminale ($n = 1024$ élèves).

- Recherche séquentielle (pire cas : élève absent ou dernier) : On compare 1024 fois. 1024 comparaisons.
- Recherche dichotomique (pire cas) : On divise par 2 à chaque étape.

$$1024 \xrightarrow{/2} 512 \quad (1)$$

$$512 \xrightarrow{/2} 256 \quad (2)$$

$$256 \xrightarrow{/2} 128 \quad (3)$$

$$128 \xrightarrow{/2} 64 \quad (4)$$

$$64 \xrightarrow{/2} 32 \quad (5)$$

$$32 \xrightarrow{/2} 16 \quad (6)$$

$$16 \xrightarrow{/2} 8 \quad (7)$$

$$8 \xrightarrow{/2} 4 \quad (8)$$

$$4 \xrightarrow{/2} 2 \quad (9)$$

$$2 \xrightarrow{/2} 1 \quad (10)$$

10 comparaisons seulement.

Ratio : $1024/10 \approx 100$ fois moins d'opérations. Pour $n = 1\,000\,000$ (fichier clients CAM-TEL), séquentielle = 1 000 000 ops, dichotomique ≈ 20 ops. 50 000 fois plus rapide.

LESAVAIS-TU ? Les moteurs de recherche et les index B-Arbres

Google, les annuaires téléphoniques d'antan, les bases de données (MySQL, Access, PostgreSQL) n'utilisent pas la dichotomie « brute » sur un tableau plat (coûteux à maintenir trié à chaque insertion/suppression). Ils utilisent des arbres de recherche équilibrés (type B-Arbre ou B+Arbre).

- L'idée reste la même : diviser pour régner (dichotomie généralisée).
- Au lieu de couper en 2, on coupe en k branches (ex : 100 à 1000) à chaque nœud.
- Hauteur de l'arbre $\approx \log_k n$. Pour 1 milliard d'enregistrements (10^9), hauteur ≈ 3 ou 4 seulement.
- Avantage majeur : insertion/suppression en $\log_k n$ sans tout décaler (contrairement au

tableau).

C'est la version « industrielle » de ce que vous apprenez ici !

6.5 Fiche méthode : rédiger proprement un algorithme (Standard Bac)

Un algorithme lisible est un algorithme noté. Voici le canevas imposé par les correcteurs du Probatoire.

MÉTHODE

Structure type d'un algorithme complet (en-tête + corps)

1. En-tête (Obligatoire) :

```
Algorithme <NomSignificatif>
// But : <Une phrase claire : "Tri par sélection croissant de T">
// Entrées : <Liste des paramètres + types + sens (entrée/sortie)>
// Sorties : <Ce qui est retourné ou affiché>
// Préconditions : <Ex : "T trié", "N > 0">
// Postconditions : <Ex : "T trié dans l'ordre croissant">
```

2. Déclaration des variables (Obligatoire) :

```
Variables
  T : Tableau[0..N-1] d'entiers // Ou "de réels", "de chaînes"
  N, i, j, temp, mil, deb, fin : Entiers
  trouve : Booléen
  x : Entier // Valeur cherchée
Début
```

3. Corps de l'algorithme (Indentation rigoureuse = 3 espaces ou 1 tab) :

- Une instruction par ligne.
- Mots-clés en majuscules : DÉBUT, FIN, SI, ALORS, SINON, FINSI, POUR, FINPOUR, TANTQUE, FINTANTQUE, RETOURNER, AFFICHER.
- Affectation : flèche ← (ou := selon habitude, mais ← est standard MINESEC).
- Commentaires explicatifs sur les lignes clés : // Invariant : T[0..i-1] trié.

4. Fin : Fin (aligné sur Début).

EXEMPLE

Modèle « Prêt à l'emploi » : Recherche Dichotomique (Standard Bac)

```
Algorithme RechercheDichotomique
```

```
// But : Rechercher x dans un tableau T trié croissant
// Entrées : T (Tableau[0..N-1] d'entiers, trié), N (Entier > 0), x (Entier)
// Sorties : position (Entier) : indice de x si trouvé, -1 sinon
// Précondition : T est trié dans l'ordre croissant
// Postcondition : Si position != -1 alors T[position] = x
```

Variables

```
T : Tableau[0..N-1] d'entiers
N, x, deb, fin, mil, position : Entiers
trouve : Booléen
```

Début

```
// Initialisation des bornes
deb <- 0
fin <- N - 1
trouve <- Faux
position <- -1
```

```
// Boucle principale : tant que l'intervalle est valide
```

```
TantQue (deb <= fin) ET (Non trouve) faire
```

```
    mil <- (deb + fin) / 2 // Division entière (arrondi inf)
```

```
    Si T[mil] = x Alors
```

```
        trouve <- Vrai
```

```
        position <- mil
```

```
    Sinon
```

```
        Si T[mil] < x Alors
```

```
            deb <- mil + 1 // On cherche dans la moitié droite
```

```
        Sinon
```

```
            fin <- mil - 1 // On cherche dans la moitié gauche
```

```
        FinSi
```

```
    FinSi
```

```
FinTantQue
```

```
AFFICHER "Position : ", position
```

```
// Ou RETOURNER position si c'est une fonction
```

```
Fin
```

EXERCICE 6 Entraînement Bac : Choix et Justification

On dispose d'un fichier contenant les notes de N élèves (N variable).

1. On veut savoir si la note 18 existe dans le fichier. Le fichier n'est pas trié. Quel algorithme choisir? Justifier en une phrase (coût + condition).
2. On doit effectuer 50 recherches successives sur ce même fichier non trié ($N = 200$). Proposer la stratégie optimale, estimer le nombre total de comparaisons (ordres de grandeur) et la comparer à 50 recherches séquentielles.
3. Écrire l'algorithme complet (selon la fiche méthode) du tri par sélection croissant sur un tableau $T[0..N-1]$. Soignez l'invariant de boucle en commentaire.

CORRIGÉ Exercice n

1. Recherche séquentielle. Justification : « Le tableau n'étant pas trié, la dichotomie est inapplicable (précondition violée). La séquentielle s'applique sans condition et coûte N comparaisons au pire, ce qui est optimal pour une unique recherche sans tri préalable. »
2. Stratégie : Trier par sélection (ou insertion) une fois, puis 50 dichotomies.
 - Coût Tri sélection : $\approx N^2/2 = 20\,000$ comparaisons.
 - Coût 50 dichotomies : $50 \times \log_2(200) \approx 50 \times 8 = 400$ comparaisons.
 - Total : $\approx 20\,400$ comparaisons.

Alternative (50 séquentielles) : $50 \times 200 = 10\,000$ comparaisons. Analyse : Ici $N = 200$ est petit. $10\,000 < 20\,400$. Contre-intuitivement, pour $N = 200$ et $q = 50$, 50 séquentielles sont plus rapides que trier + 50 dichotomies ! Le seuil de rentabilité du tri ($N^2/2$) n'est pas encore atteint. (Point de vigilance : toujours estimer les coûts avant de décider).

3. Algorithme TriSelection (Standard Bac) :

Algorithme TriSelection

```
// But : Trier le tableau T dans l'ordre croissant par sélection
// Entrées/Sortie : T (Tableau[0..N-1] d'entiers), N (Entier > 0)
// Précondition : N > 0
// Postcondition : T est trié croissant ; permutation de l'initial
```

Variables

```
T : Tableau[0..N-1] d'entiers
N, i, j, minIdx, temp : Entiers
```

Début

```
Pour i <- 0 à N - 2 faire           // Invariant : T[0..i-1] trié & <= tout élément dans T[i..N-1]
  minIdx <- i
  // Recherche du minimum dans la partie non triée T[i..N-1]
  Pour j <- i + 1 à N - 1 faire
    Si T[j] < T[minIdx] Alors
      minIdx <- j
  FinSi
FinPour
```

```
// Échange T[i] et T[minIdx] si nécessaire
Si minIdx <> i Alors
  temp <- T[i]
  T[i] <- T[minIdx]
  T[minIdx] <- temp
FinSi
FinPour
Fin
```

À RETENIR**Check-list finale avant de rendre votre copie (Probatoire)**

- Justification écrite pour chaque choix d'algorithme (condition + coût).
- Indices : Départ à 0 (tableau $0..N-1$) ou 1 (selon énoncé) → cohérence totale dans les bornes.
- Variable temporaire pour tout échange.
- Division entière pour le milieu en dichotomie.
- Indentation parfaite (3 espaces) : Début, Fin, Si, FinSi, Pour, FinPour, TantQue, FinTantQue alignés verticalement par blocs.
- Commentaires : But, Pré/Post-condition, Invariant de boucle (pour les tris).

7 ACTIVITÉ D'INTÉGRATION

8 ALGORITHMES DE TRI ET DE RECHERCHE

8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- **exécuter** à la main les algorithmes de **tri par sélection** et de **tri par insertion** sur un tableau de nombres, en rédigeant un tableau d'étapes complet ;
- **écrire** un algorithme de tri simple sur un tableau à valeurs numériques, paramétré par la taille n du tableau ;
- **décrire** et **exécuter** les recherches **séquentielle** et **dichotomique**, en comptant le nombre de comparaisons effectuées ;
- **comparer** les coûts des deux méthodes de recherche et conclure sur la plus économique ;
- travailler en groupe, répartir les tâches et justifier oralement ses résultats devant la classe.

8.2 Situation

Le Lycée Bilingue de Nkol-Eton organise chaque année son grand concours d'éloquence « Miss et Mister Culture ». Cette année, huit candidats se sont présentés devant un jury présidé par le proviseur. À la fin des prestations, le jury a relevé les notes sur 20 dans l'ordre de passage des candidats :

Candidat	C1	C2	C3	C4	C5	C6	C7	C8
Note	14	8	17	11	6	15	9	12

Le tableau des notes, dans l'ordre de passage, est donc :

$$T = [14 ; 8 ; 17 ; 11 ; 6 ; 15 ; 9 ; 12]$$

Pour proclamer les résultats, le proviseur exige un **classement par ordre croissant** des notes. De plus, une rumeur circule dans la cour : un élève affirme que la note **15** n'a jamais été attribuée. Le proviseur te demande, à toi la classe de Première TI, de :

1. trier le tableau des notes par ordre croissant, d'abord avec le **tri par sélection**, puis vérifier le résultat avec le **tri par insertion** ;
2. rédiger l'**algorithme** du tri que ton groupe aura jugé le plus simple à programmer, en le généralisant à un tableau de taille n ;
3. déterminer si la note **15** figure dans le tableau, d'abord par **recherche séquentielle** sur le tableau initial (non trié), puis par **recherche dichotomique** sur le tableau trié, en **comptant les comparaisons** dans les deux cas ;

4. rédiger l'**algorithme de recherche dichotomique** généralisé et conclure sur la méthode la plus économique.

8.3 Consignes

Travail en groupes de 4 élèves. Chaque groupe rédige un rapport qui sera présenté au tableau.

1. **Tri par sélection.** Exécute le tri par sélection sur le tableau T . Rédige le **tableau d'étapes** : pour chaque passage i (de 1 à $n-1$), indique la position du minimum trouvé, l'échange effectué et l'état du tableau après l'échange. Combien d'échanges as-tu effectués au total?
2. **Vérification par le tri par insertion.** Exécute maintenant le tri par insertion sur le même tableau initial. Rédige le tableau d'étapes : pour chaque passage i (de 2 à n), indique la valeur insérée et l'état du tableau après insertion. Le résultat final est-il identique à celui de la question 1?
3. **Écriture de l'algorithme de tri.** Choisis l'un des deux tris et rédige son algorithme complet en pseudo-code (déclaration du tableau, variables, boucles, échange). Ton algorithme doit fonctionner pour tout tableau de n notes.
4. **Recherche séquentielle.** Sur le tableau initial (non trié), exécute une recherche séquentielle de la valeur 15. Note chaque comparaison effectuée. La valeur est-elle trouvée? En combien de comparaisons? Combien de comparaisons aurait-il fallu si la note cherchée avait été 10 (valeur absente)?
5. **Recherche dichotomique.** Sur le tableau trié obtenu à la question 1, exécute une recherche dichotomique de la valeur 15. À chaque étape, indique les bornes `debut` et `fin`, la position `milieu` et la comparaison effectuée. En combien de comparaisons la valeur est-elle trouvée?
6. **Écriture de l'algorithme de recherche.** Rédige l'algorithme complet de la recherche dichotomique en pseudo-code, paramétré par la taille n du tableau.
7. **Conclusion.** Compare le nombre de comparaisons des questions 4 et 5. Quelle méthode est la plus économique? À quelle condition peut-on l'utiliser? Rédige un paragraphe de conclusion pour le proviseur.

8.4 Ressources à mobiliser

À RETENIR Rappels de la leçon

- Tri par sélection : pour chaque position i de 1 à $n-1$, on cherche le **minimum** de la partie non triée $T[i..n]$ et on l'échange avec $T[i]$. Au plus $n-1$ échanges. Complexité en comparaisons : $\frac{n(n-1)}{2} \in O(n^2)$.
- Tri par insertion : pour chaque position i de 2 à n , on prend la valeur $T[i]$ (clé) et on l'**insère** à sa place dans la partie déjà triée $T[1..i-1]$, en décalant les éléments plus grands vers la droite. Complexité en comparaisons : $O(n^2)$ au pire, $O(n)$ au meilleur (ta-

bleau déjà trié).

- Recherche séquentielle : on compare la valeur cherchée à chaque élément, du premier au dernier, jusqu'à trouver ou épuiser le tableau. Coût : au plus n comparaisons. Fonctionne sur tout tableau (trié ou non).
- Recherche dichotomique : sur un tableau **trié**, on compare la valeur cherchée à l'élément du **milieu** ; selon le résultat, on élimine la moitié gauche ou droite du tableau, et on recommence. Coût : au plus $\lfloor \log_2(n) \rfloor + 1$ comparaisons. Condition impérative : le tableau doit être trié au préalable.

MÉTHODE Exécuter un tri par sélection sur un tableau $T[1..n]$

1. Pour i allant de 1 à $n - 1$:
 - (a) Poser $\text{posMin} \leftarrow i$.
 - (b) Pour j allant de $i + 1$ à n : si $T[j] < T[\text{posMin}]$ alors $\text{posMin} \leftarrow j$.
 - (c) Si $\text{posMin} \neq i$ alors échanger $T[i]$ et $T[\text{posMin}]$ (via une variable temporaire).
2. Le tableau est trié par ordre croissant.

MÉTHODE Exécuter un tri par insertion sur un tableau $T[1..n]$

1. Pour i allant de 2 à n :
 - (a) $\text{cle} \leftarrow T[i]$; $j \leftarrow i - 1$.
 - (b) Tant que $j \geq 1$ et $T[j] > \text{cle}$ faire : $T[j + 1] \leftarrow T[j]$; $j \leftarrow j - 1$.
 - (c) $T[j + 1] \leftarrow \text{cle}$.
2. Le tableau est trié par ordre croissant.

MÉTHODE Exécuter une recherche dichotomique (tableau trié $T[1..n]$, valeur x)

1. $\text{debut} \leftarrow 1$; $\text{fin} \leftarrow n$; $\text{trouve} \leftarrow \text{Faux}$.
2. Tant que $\text{debut} \leq \text{fin}$ et $\text{trouve} = \text{Faux}$ faire :
 - (a) $\text{milieu} \leftarrow (\text{debut} + \text{fin}) \text{ DIV } 2$.
 - (b) Si $T[\text{milieu}] = x$ alors $\text{trouve} \leftarrow \text{Vrai}$.

- (c) Sinon si $T[\text{milieu}] < x$ alors $\text{debut} \leftarrow \text{milieu} + 1$.
- (d) Sinon $\text{fin} \leftarrow \text{milieu} - 1$.
3. Si trouve alors $\text{position} = \text{milieu}$ sinon « absent ».

PROPRIÉTÉ Comparaison des coûts de recherche

Pour un tableau de taille n :

- Recherche séquentielle : $C_{\text{seq}}(n) \leq n$ comparaisons. Complexité linéaire $O(n)$.
- Recherche dichotomique : $C_{\text{dicho}}(n) \leq \lfloor \log_2 n \rfloor + 1$ comparaisons. Complexité logarithmique $O(\log n)$.

La dichotomique est asymptotiquement plus rapide, mais exige un tableau trié. Le tri préalable coûte $O(n^2)$ (sélection/insertion) ou $O(n \log n)$ (tri rapide, fusion – vus en Terminale). La dichotomique devient rentable si l'on effectue plusieurs recherches sur le même tableau trié.

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Corrigé complet de l'activité

Question 1. Tri par sélection de $T = [14; 8; 17; 11; 6; 15; 9; 12]$ ($n = 8$).

À chaque passage i , on cherche le minimum de la partie non triée $T[i..8]$ et on le place en position i .

Passage i	Minimum trouvé (position)	Échange	Tableau après échange
1	6 (pos. 5)	$T[1] \leftrightarrow T[5]$	[6 ; 8 ; 17 ; 11 ; 14 ; 15 ; 9 ; 12]
2	8 (pos. 2)	aucun (déjà en place)	[6 ; 8 ; 17 ; 11 ; 14 ; 15 ; 9 ; 12]
3	9 (pos. 7)	$T[3] \leftrightarrow T[7]$	[6 ; 8 ; 9 ; 11 ; 14 ; 15 ; 17 ; 12]
4	11 (pos. 4)	aucun	[6 ; 8 ; 9 ; 11 ; 14 ; 15 ; 17 ; 12]
5	12 (pos. 8)	$T[5] \leftrightarrow T[8]$	[6 ; 8 ; 9 ; 11 ; 12 ; 15 ; 17 ; 14]
6	14 (pos. 8)	$T[6] \leftrightarrow T[8]$	[6 ; 8 ; 9 ; 11 ; 12 ; 14 ; 17 ; 15]
7	15 (pos. 8)	$T[7] \leftrightarrow T[8]$	[6 ; 8 ; 9 ; 11 ; 12 ; 14 ; 15 ; 17]

On a effectué 5 échanges (les passages 2 et 4 n'ont nécessité aucun échange car le minimum était déjà en place). Le tableau trié par ordre croissant est :

$$T_{\text{trié}} = [6 ; 8 ; 9 ; 11 ; 12 ; 14 ; 15 ; 17]$$

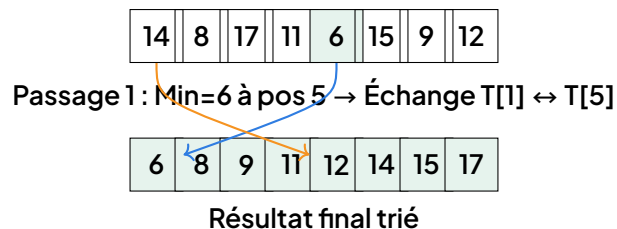


Illustration du premier passage du tri par sélection : le minimum (6) rejoint la tête du tableau.

Question 2. Tri par insertion du tableau initial [14; 8; 17; 11; 6; 15; 9; 12].

Passage i	Valeur insérée (clé)	Tableau après insertion
2	8	[8; 14; 17; 11; 6; 15; 9; 12]
3	17	[8; 14; 17; 11; 6; 15; 9; 12]
4	11	[8; 11; 14; 17; 6; 15; 9; 12]
5	6	[6; 8; 11; 14; 17; 15; 9; 12]
6	15	[6; 8; 11; 14; 15; 17; 9; 12]
7	9	[6; 8; 9; 11; 14; 15; 17; 12]
8	12	[6; 8; 9; 11; 12; 14; 15; 17]

Le résultat final est identique : [6; 8; 9; 11; 12; 14; 15; 17]. Les deux tris donnent le même tableau trié, ce qui valide le travail de la question 1. Seul le chemin pour y arriver diffère : sélection « tire » les petits vers la gauche, insertion « glisse » les grands vers la droite.

Question 3. Algorithme du tri par sélection (généralisé à n éléments).

```

Algorithme TriSelection
Variable
  T : tableau[1..n] d'entiers
  i, j, posMin, temp : entier
  n : entier // taille du tableau
Début
  // Supposons n déjà connu ou lu au clavier
  Pour i de 1 à n-1 Faire
    posMin ← i
    Pour j de i+1 à n Faire
      Si T[j] < T[posMin] Alors
        posMin ← j
      FinSi
    FinPour
    // Échange de T[i] et T[posMin] si nécessaire
    Si posMin <> i Alors
      temp ← T[i]
      T[i] ← T[posMin]
      T[posMin] ← temp
    FinSi
  FinPour
Fin

```

Question 4. Recherche séquentielle de 15 dans le tableau initial [14; 8; 17; 11; 6; 15; 9; 12].

Comparaison	Élément lu	Résultat
1	$T[1] = 14$	$14 \neq 15$, on continue
2	$T[2] = 8$	$8 \neq 15$, on continue
3	$T[3] = 17$	$17 \neq 15$, on continue
4	$T[4] = 11$	$11 \neq 15$, on continue
5	$T[5] = 6$	$6 \neq 15$, on continue
6	$T[6] = 15$	$15 = 15$, trouvé!

La note 15 est trouvée en 6 comparaisons. Si l'on avait cherché la note 10 (absente), il aurait fallu parcourir tout le tableau : 8 comparaisons pour conclure à l'absence.

Question 5. Recherche dichotomique de 15 dans le tableau trié [6; 8; 9; 11; 12; 14; 15; 17].

Étape	debut	fin	milieu	Comparaison et décision
1	1	8	4	$T[4] = 11 < 15$: on garde la moitié droite, $\text{debut} \leftarrow 5$
2	5	8	6	$T[6] = 14 < 15$: on garde la droite, $\text{debut} \leftarrow 7$
3	7	8	7	$T[7] = 15 = 15$: trouvé!

La note 15 est trouvée en seulement 3 comparaisons, contre 6 pour la recherche séquentielle.



Visualisation de la recherche dichotomique : l'intervalle de recherche est divisé par deux à chaque étape.

Question 6. Algorithme de la recherche dichotomique (généralisé à n éléments).

Algorithme RechercheDichotomique

Variable

T : tableau[1..n] d'entiers // Tableau TRIÉ dans l'ordre croissant
x, debut, fin, milieu : entier
trouve : booléen
n : entier

Début

Écrire("Entrer la taille du tableau n :")
Lire(n)

// Remplissage du tableau T[1..n] supposé déjà fait et TRIÉ

Écrire("Entrer la valeur cherchée x :")

Lire(x)

debut $\leftarrow$ 1

fin $\leftarrow$ n

trouve $\leftarrow$ Faux

Tant que (debut $\leq$ fin) ET (trouve = Faux) Faire

 milieu $\leftarrow$ (debut + fin) DIV 2

 Si T[milieu] = x Alors

 trouve $\leftarrow$ Vrai

 Sinon

 Si T[milieu] < x Alors

 debut $\leftarrow$ milieu + 1

 Sinon

 fin $\leftarrow$ milieu - 1

 FinSi

FinSi

```

FinTantQue
Si trouve = Vrai Alors
    Écrire("Valeur trouvée à la position ", milieu)
Sinon
    Écrire("Valeur absente du tableau")
FinSi
Fin

```

Question 7. Conclusion.

Pour trouver la note 15, la recherche séquentielle a demandé 6 comparaisons et la recherche dichotomique seulement 3. L'écart grandit très vite avec la taille du tableau : pour $n = 1024$ notes, la recherche séquentielle peut exiger jusqu'à 1024 comparaisons, alors que la dichotomique n'en demande au plus que 10 (car $2^{10} = 1024$). La recherche dichotomique est donc la plus économique en temps d'exécution, à condition impérative que le tableau soit trié au préalable.

Message au proviseur : « La note 15 a bien été attribuée (au candidat C6) ; le classement croissant des notes est : 6 ; 8 ; 9 ; 11 ; 12 ; 14 ; 15 ; 17. »

ATTENTION Pièges fréquents à éviter pendant l'activité

- Dans le tri par sélection, ne pas confondre la valeur du minimum et sa position : c'est la position ($posMin$) qu'il faut mémoriser pour pouvoir échanger.
- Ne jamais lancer une recherche dichotomique sur le tableau initial non trié : elle pourrait conclure à tort que 15 est absent (ex : 15 est à l'indice 6, mais le milieu initial est 4 $\rightarrow 11 < 15$, on part à droite, on rate le 15 qui était à gauche du milieu dans le désordre).
- Dans la recherche dichotomique, penser à mettre à jour $debut$ ou fin avec $milieu + 1$ ou $milieu - 1$ (et non $milieu$), sinon la boucle peut devenir infinie si l'élément n'est pas au milieu.
- L'échange de deux cases exige une variable temporaire : écrire $T[i] \leftarrow T[posMin]$ puis $T[posMin] \leftarrow T[i]$ écrase définitivement la première valeur.
- Dans l'algorithme généralisé, ne pas « coder en dur » la valeur 8 : utiliser la variable n pour les bornes des boucles ($1 \text{ à } n-1, i+1 \text{ à } n, fin \leftarrow n$).

8.6 Bilan de l'activité

Cette activité t'a permis de mettre en évidence plusieurs faits essentiels de la leçon :

- **Deux algorithmes différents peuvent produire le même résultat.** Le tri par sélection et le tri par insertion aboutissent tous deux au tableau trié [6 ; 8 ; 9 ; 11 ; 12 ; 14 ; 15 ; 17], mais par des démarches différentes : l'un sélectionne le minimum global de la zone non triée, l'autre insère chaque nouvel élément dans la zone triée. Vérifier un tri par un autre tri est une excellente pratique de contrôle (test croisé).

- **Le tri prépare le terrain à une recherche efficace.** Sur un petit tableau ($n = 8$), la recherche séquentielle reste acceptable (6 comparaisons ici). Mais dès que le tableau grossit, la recherche dichotomique devient imbattable : son coût croît comme $\log_2(n)$ et non comme n .
- **Toute méthode a ses conditions d'application.** La dichotomie n'est valable que sur un tableau trié : c'est le « prix à payer » (un tri préalable en $O(n^2)$ ici) pour gagner en rapidité ensuite ($O(\log n)$ par recherche). En informatique, on choisit toujours un algorithme en fonction du contexte : taille des données, données triées ou non, nombre de recherches à effectuer, mémoire disponible.
- **Compter les opérations, c'est mesurer l'efficacité.** Tu as découvert, en comptant les comparaisons, la démarche qui mènera en Terminale à la notion formelle de **complexité algorithmique** (notations O , Ω , θ).
- **Généralisation (paramétrisation).** Un algorithme professionnel ne contient pas de constantes magiques (comme le 8 du nombre de candidats) mais des variables (n) pour traiter n'importe quelle taille de problème.

Tu es désormais capable, comme un vrai technicien en informatique, d'organiser des données (tri) et de les exploiter (recherche) en choisissant la méthode la plus économique : une compétence directement mobilisable pour gérer les notes d'une classe, les résultats d'un concours, le stock d'une boutique à Douala ou la base d'abonnés de CAMTEL.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 ALGORITHMES DE TRI ET DE RECHERCHE

10.1 Énumérer les algorithmes de tri

MÉTHODE Citer et reconnaître les algorithmes de tri classiques

Pour répondre à la question « Énumérez quelques algorithmes de tri » :

1. Cite au moins trois algorithmes, avec leur principe en une phrase (c'est ce qui rapporte tous les points).
2. Les algorithmes à connaître parfaitement : **tri par sélection** (on cherche le plus petit élément et on le place en tête), **tri par insertion** (on insère chaque élément à sa place dans la partie déjà triée), **tri à bulles** (on compare et échange les éléments adjacents, les plus grands « remontent » comme des bulles).
3. Précise toujours qu'un tri s'effectue sur un tableau et produit un tableau ordonné (croissant ou décroissant).
4. Si la question demande une comparaison, indique que ces trois tris ont une complexité en $O(n^2)$: le nombre de comparaisons est de l'ordre de $\frac{n(n-1)}{2}$ pour un tableau de n éléments.

EXERCICE RÉSOLU 8 Énumération des algorithmes de tri

Énoncé : Le directeur du cybercafé « Espace Net » de Bafoussam veut classer les factures du mois par montant croissant. (a) Énumérez trois algorithmes de tri en précisant le principe de chacun. (b) Combien de comparaisons, au maximum, un tri de type sélection effectue-t-il sur un tableau de 10 factures ?

SOLUTION

(a) Trois algorithmes de tri classiques :

- Tri par sélection : à chaque étape, on recherche le plus petit élément de la partie non triée et on l'échange avec le premier élément de cette partie.
- Tri par insertion : on prend les éléments un par un et on les insère à leur place dans la partie déjà triée du tableau (comme on trie des cartes en main).

- Tri à bulles : on parcourt le tableau en comparant les éléments adjacents et en les échangeant s'ils sont dans le mauvais ordre ; à chaque passage, le plus grand élément « remonte » en fin de tableau.

(b) Pour un tableau de $n = 10$ éléments, le tri par sélection effectue :

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} = \frac{10 \times 9}{2} = 45 \text{ comparaisons.}$$

Conclusion : le tri des 10 factures par sélection nécessite au maximum 45 comparaisons.

10.2 Exécuter le tri par sélection

MÉTHODE Exécuter à la main un tri par sélection (ordre croissant)

1. Recopie le tableau initial et numérote les cases de 1 à n (ou de 0 à $n-1$ selon l'énoncé — respecte la convention de l'énoncé).
2. À l'étape i (pour i allant de 1 à $n-1$) : cherche la position du minimum dans la partie non triée (cases i à n).
3. Échange cet élément avec celui de la case i . Souligne ou encadre la partie déjà triée.
4. Présente le résultat sous forme de tableau d'étapes : une ligne par étape, avec le minimum trouvé et l'échange effectué.
5. Vérifie à la fin que le tableau est bien croissant et qu'aucune valeur n'a disparu ni été dupliquée.

Réflexe : il y a exactement $n-1$ étapes (la dernière case est automatiquement en place).

EXERCICE RÉSOLU 9 Tri par sélection des notes d'une classe

Énoncé : Les notes d'informatique de 6 élèves de Première TI sont rangées dans le tableau $T : T = [12, 5, 17, 8, 14, 9]$. Exécutez le tri par sélection (ordre croissant) en donnant l'état du tableau à la fin de chaque étape.

SOLUTION

On applique la méthode : à l'étape i , on cherche le minimum dans $T[i..6]$ et on l'échange avec $T[i]$.

Étape	Minimum (position)	Échange	Tableau résultant
Départ	—	—	[12, 5, 17, 8, 14, 9]
$i = 1$	5 (case 2)	$T[1] \leftrightarrow T[2]$	[5, 12, 17, 8, 14, 9]
$i = 2$	8 (case 4)	$T[2] \leftrightarrow T[4]$	[5, 8, 17, 12, 14, 9]
$i = 3$	9 (case 6)	$T[3] \leftrightarrow T[6]$	[5, 8, 9, 12, 14, 17]
$i = 4$	12 (case 4)	aucun (déjà en place)	[5, 8, 9, 12, 14, 17]
$i = 5$	14 (case 5)	aucun (déjà en place)	[5, 8, 9, 12, 14, 17]

Après $n - 1 = 5$ étapes, la case 6 contient forcément le maximum. Conclusion : le tableau trié est [5, 8, 9, 12, 14, 17].

10.3 Exécuter le tri par insertion

MÉTHODE Exécuter à la main un tri par insertion (ordre croissant)

1. Considère la première case comme une zone déjà triée (un seul élément est toujours trié).
2. À l'étape i (de 2 à n) : mémorise la valeur $x = T[i]$ dans une variable temporaire (la **clé**).
3. Décale vers la droite tous les éléments de la zone triée qui sont strictement plus grands que x .
4. Insère x dans le trou ainsi créé.
5. Dessine le tableau après chaque insertion, en séparant visuellement la zone triée de la zone non triée (par exemple avec une barre verticale).

Réflexe : ne jamais écraser $T[i]$ avant de l'avoir sauvegardé dans la variable clé — c'est l'erreur classique.

EXERCICE RÉSOLU 10 Tri par insertion des prix d'une boutique

Énoncé : La boutique scolaire du lycée range les prix (en FCFA) de 5 articles dans le tableau $P = [250, 100, 400, 150, 300]$. Exécutez le tri par insertion (ordre croissant) en détaillant chaque étape.

SOLUTION

On note la zone triée à gauche de la barre |.

Étape 1 : clé $x = P[2] = 100$. Dans la zone triée [250], $250 > 100$ donc on décale 250 à droite

et on insère 100 en case 1 : [100, 250 | 400, 150, 300].

Étape 2 : clé $x = P[3] = 400$. Dans [100, 250], aucun élément n'est supérieur à 400 : insertion sans décalage : [100, 250, 400 | 150, 300].

Étape 3 : clé $x = P[4] = 150$. On décale 400 puis 250 (tous deux > 150); $100 < 150$ on s'arrête. Insertion en case 2 : [100, 150, 250, 400 | 300].

Étape 4 : clé $x = P[5] = 300$. On décale 400 (seul élément > 300); insertion en case 4 : [100, 150, 250, 300, 400].

Conclusion : le tableau trié est [100, 150, 250, 300, 400] ; les prix sont classés du moins cher au plus cher.

10.4 Écrire un algorithme de tri simple

MÉTHODE Rédiger l'algorithme du tri par sélection

1. Commence par l'en-tête : nom de l'algorithme, déclaration des variables (tableau T de n réels, indices $i, j, imin$, variable d'échange aux).
2. Écris la boucle externe : Pour i de 1 à $n - 1$.
3. À l'intérieur : initialise $imin \leftarrow i$, puis la boucle interne Pour j de $i + 1$ à n qui met à jour $imin$ si $T[j] < T[imin]$.
4. Après la boucle interne, effectue l'échange en trois instructions avec la variable auxiliaire : $aux \leftarrow T[i]$; $T[i] \leftarrow T[imin]$; $T[imin] \leftarrow aux$.
5. Termine proprement (Fin Pour, Fin). Relis-toi en testant mentalement sur un tableau de 3 éléments.

EXERCICE RÉSOLU 11 Algorithme de tri des montants de factures

Énoncé : Écrivez un algorithme qui trie par sélection, en ordre croissant, un tableau T de n montants de factures (réels positifs) déjà saisis.

SOLUTION

```

Algorithme TriSelection
Variables
  T : tableau[1..n] de Reel
  n, i, j, imin : Entier
  aux : Reel
Debut
  Pour i de 1 a n-1 Faire
    imin ← i

```

```
Pour j de i+1 à n Faire
    Si T[j] < T[imin] Alors
        imin <- j
    FinSi
FinPour
aux <- T[i]
T[i] <- T[imin]
T[imin] <- aux
FinPour
Fin
```

Justification : la boucle externe fixe la case i à remplir ; la boucle interne trouve la position $imin$ du plus petit élément de la partie non triée ; l'échange en trois instructions place ce minimum en case i . Après $n - 1$ passages, le tableau est entièrement trié en ordre croissant. Conclusion : l'algorithme ci-dessus trie correctement le tableau des factures, avec un coût de $\frac{n(n-1)}{2}$ comparaisons.

10.5 Décrire les recherches séquentielle et dichotomique

MÉTHODE Décrire le principe d'une recherche et choisir la bonne

1. Recherche séquentielle : on parcourt le tableau case par case depuis le début, en comparant chaque élément à la valeur cherchée ; on s'arrête dès qu'on la trouve (ou en fin de tableau). Aucune condition sur le tableau.
2. Recherche dichotomique : on compare la valeur cherchée à l'élément du milieu du tableau ; selon le résultat, on élimine la moitié du tableau et on recommence sur la moitié restante. Condition impérative : le tableau doit être trié.
3. Pour choisir : tableau non trié $\Rightarrow$ séquentielle ; tableau trié $\Rightarrow$ dichotomique, bien plus rapide : au plus $\log_2(n)$ comparaisons contre n pour la séquentielle.
4. Rédige toujours la condition d'application dans ta description : c'est souvent un point de la question.

EXERCICE RÉSOLU 12 Comparaison des deux recherches

Énoncé : Le registre d'une bibliothèque scolaire contient 1 024 ouvrages. (a) Décrivez le principe de la recherche séquentielle et celui de la recherche dichotomique. (b) Quel est le nombre maximal de comparaisons pour retrouver un ouvrage avec chaque méthode, si le registre est trié par titre ?

SOLUTION

(a) Recherche séquentielle : on examine les ouvrages un à un, du premier au dernier, jusqu'à trouver le titre cherché ou épuiser la liste ; elle fonctionne sur tout tableau, trié ou non. Recherche dichotomique : le registre étant trié, on le « coupe en deux » : on compare le titre cherché à l'ouvrage médian ; s'il est plus petit on ne garde que la première moitié, s'il est plus grand la seconde ; on recommence jusqu'à trouver l'ouvrage ou obtenir un intervalle vide. Condition : le tableau doit être trié.

(b) Recherche séquentielle : au pire on parcourt tout le registre, soit 1 024 comparaisons. Recherche dichotomique : à chaque comparaison la zone de recherche est divisée par 2 ; le nombre maximal de comparaisons est le plus petit entier k tel que $2^k \geq n$:

$$2^{10} = 1\,024 \implies k = \log_2(1\,024) = 10.$$

Conclusion : au maximum 1 024 comparaisons en séquentiel contre seulement 10 en dichotomique : la recherche dichotomique est ici environ 100 fois plus efficace, grâce au tri préalable du registre.

10.6 Écrire un algorithme de recherche dans un tableau

MÉTHODE Rédiger une recherche séquentielle

1. Déclare : le tableau T , sa taille n , la valeur cherchée x , l'indice i , et un booléen `trouve` (ou une position initialisée à 0).
2. Initialise $i \leftarrow 1$ et `trouve` $\leftarrow$ Faux.
3. Écris une boucle Tant que ($i \leq n$ ET `trouve` = Faux) : si $T[i] = x$ alors `trouve` $\leftarrow$ Vrai (et mémorise la position), sinon $i \leftarrow i + 1$.
4. Après la boucle, affiche la position si trouvé, sinon le message « élément absent ».
5. Réflexe : la double condition dans le Tant que évite de dépasser la fin du tableau quand l'élément est absent — ne l'oublie jamais.

EXERCICE RÉSOLU 13 Recherche d'un élève par matricule

Énoncé : Écrivez un algorithme qui recherche un matricule x dans un tableau T de n matricules d'élèves (non trié) et affiche sa position ou un message d'absence.

SOLUTION

Algorithme RechercheSequentielle

Variables

T : tableau[1..n] de Entier

n, i, x, pos : Entier

trouve : Booleen

Debut

Ecrire("Entrer le matricule cherche : ")

Lire(x)

i ← 1

trouve ← Faux

pos ← 0

Tant que (i ≤ n) ET (trouve = Faux) Faire

Si T[i] = x Alors

trouve ← Vrai

pos ← i

Sinon

i ← i + 1

FinSi

FinTantQue

Si trouve = Vrai Alors

Ecrire("Matricule trouve a la position", pos)

Sinon

Ecrire("Matricule absent du tableau")

FinSi

Fin

Justification : le tableau n'étant pas trié, la recherche dichotomique est interdite ; on parcourt donc séquentiellement. La condition $(i \leq n)$ ET $(trouve = \text{Faux})$ garantit l'arrêt dès la première occurrence trouvée ou en fin de tableau sans débordement. Conclusion : l'algorithme affiche la position du matricule s'il existe, et « Matricule absent du tableau » sinon.

10.7 Exécuter une recherche dichotomique

MÉTHODE

Exécuter à la main une recherche dichotomique

1. Vérifie d'abord que le tableau est trié (sinon, dis-le et refuse d'appliquer la méthode).
2. Pose deux bornes : $deb \leftarrow 1$, $fin \leftarrow n$. Calcule le milieu : $milieu \leftarrow \left\lfloor \frac{deb + fin}{2} \right\rfloor$ (partie entière).
3. Compare $T[milieu]$ à la valeur x : si égalité, c'est terminé ; si $x < T[milieu]$, alors $fin \leftarrow milieu - 1$; si $x > T[milieu]$, alors $deb \leftarrow milieu + 1$.
4. Recommence tant que $deb \leq fin$. Si $deb > fin$, l'élément est absent.
5. Présente chaque itération dans un tableau : $deb, fin, milieu, T[milieu]$, décision.

EXERCICE RÉSOLU 14 Recherche dichotomique d'une note

Énoncé : Soit le tableau trié $T = [4, 7, 9, 12, 15, 18, 20]$. Exécutez la recherche dichotomique de la valeur $x = 15$ en détaillant chaque itération, puis indiquez ce qui se passerait pour $x = 10$.

SOLUTION

Le tableau est trié en ordre croissant : la dichotomie est applicable. $n = 7$.

Recherche de 15 :

Itération	<i>deb</i>	<i>fin</i>	<i>milieu</i>	$T[\textit{milieu}]$	Décision
1	1	7	4	12	$15 > 12 \Rightarrow \textit{deb} \leftarrow 5$
2	5	7	6	18	$15 < 18 \Rightarrow \textit{fin} \leftarrow 5$
3	5	5	5	15	$15 = 15$: trouvé

La valeur 15 est trouvée à la position 5 en seulement 3 comparaisons (au lieu de 5 en séquentiel).

Recherche de 10 : itération 1 : $\textit{milieu} = 4$, $T[4] = 12$, $10 < 12 \Rightarrow \textit{fin} \leftarrow 3$; itération 2 : $\textit{milieu} = 2$, $T[2] = 7$, $10 > 7 \Rightarrow \textit{deb} \leftarrow 3$; itération 3 : $\textit{milieu} = 3$, $T[3] = 9$, $10 > 9 \Rightarrow \textit{deb} \leftarrow 4$. Or $\textit{deb} = 4 > \textit{fin} = 3$: la zone de recherche est vide. Conclusion : 10 se trouve en position 5 ; la valeur 10 est absente du tableau.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Appliquer la dichotomie sur un tableau non trié : c'est l'erreur la plus grave. Écris toujours la condition d'application (« le tableau étant trié... ») avant de commencer l'exécution.
2. Écraser la clé dans le tri par insertion : si tu ne sauvegardes pas $T[i]$ dans une variable avant de décaler, la valeur est perdue. Exécute mentalement l'étape sur un exemple de 3 cases pour vérifier.
3. Échanger en deux instructions dans le tri par sélection ($T[i] \leftarrow T[\textit{imin}]$ puis $T[\textit{imin}] \leftarrow T[i]$) : la première valeur est écrasée. L'échange exige trois instructions avec une variable auxiliaire.
4. Oublier la double condition d'arrêt dans la recherche séquentielle : sans le test $i \leq n$, l'algorithme lit au-delà du tableau quand l'élément est absent (débordement).
5. Se tromper dans le calcul du milieu ou des bornes : $\textit{milieu} = \lfloor (\textit{deb} + \textit{fin})/2 \rfloor$ (partie entière), et on met à jour avec $\textit{milieu} + 1$ ou $\textit{milieu} - 1$ (jamais $\textit{milieu}$ tout seul, sinon boucle infinie). Une erreur de borne fausse toute l'exécution.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 7 QCM : tri et recherche

Pour chaque question, une seule réponse est correcte. Entoure la bonne lettre.

1. Le principe du tri par sélection consiste à :

- a) insérer chaque élément à sa place dans la partie déjà triée ;
- b) rechercher le plus petit élément de la partie non triée et l'échanger avec le premier élément de cette partie ;
- c) comparer les éléments deux à deux et les permuter s'ils sont mal ordonnés ;
- d) diviser le tableau en deux moitiés.

2. La recherche dichotomique ne peut s'appliquer que si :

- a) le tableau contient au moins 100 éléments ;
- b) les éléments du tableau sont des nombres ;
- c) le tableau est trié ;
- d) le tableau ne contient pas de doublons.

3. Pour rechercher un élément dans un tableau de n éléments, la recherche séquentielle effectuée, dans le pire des cas :

- a) 1 comparaison ; b) $\log_2 n$ comparaisons ; c) n comparaisons ; d) n^2 comparaisons.

4. Dans la recherche dichotomique, si la valeur cherchée est strictement supérieure à l'élément du milieu, alors :

- a) on poursuit la recherche dans la moitié gauche du tableau ;
- b) on poursuit la recherche dans la moitié droite du tableau ;
- c) la recherche est terminée : la valeur est absente ;
- d) on recommence depuis le début du tableau.

EXERCICE 8 Vrai ou faux ? Justifie

Réponds par Vrai ou Faux, puis justifie ta réponse en une ou deux phrases.

- 1. Le tri par insertion consiste à prendre les éléments un par un et à les placer à leur position correcte dans la partie déjà triée du tableau.

2. La recherche séquentielle exige que le tableau soit trié par ordre croissant.
3. Dans le tri par sélection d'un tableau de n éléments, le nombre de passages (tours de boucle principale) est $n - 1$.
4. La recherche dichotomique est toujours plus lente que la recherche séquentielle.

EXERCICE 9 Définitions

Définis clairement, en deux ou trois phrases chacun, les notions suivantes :

1. algorithme de tri ;
2. recherche séquentielle ;
3. recherche dichotomique ;
4. clef de tri.

EXERCICE 10 Calculs directs

1. Un tableau contient $n = 10$ éléments. Combien de comparaisons la recherche séquentielle effectue-t-elle dans le pire des cas ?
2. Un tableau trié contient $n = 128$ éléments. Sachant qu'à chaque étape la recherche dichotomique divise la zone de recherche par 2, combien d'étapes au maximum sont nécessaires pour conclure ? (On rappelle que $2^7 = 128$.)
3. Même question pour un tableau de 1000 éléments triés, sachant que $2^{10} = 1024$. Compare avec le nombre maximal de comparaisons d'une recherche séquentielle sur ce même tableau.

11.2 Je m'entraîne

EXERCICE 11 Exécution du tri par sélection

On considère le tableau d'entiers suivant, indicé à partir de 1 :

Indice	1	2	3	4	5
Valeur	25	13	40	7	18

Exécute pas à pas l'algorithme du tri par sélection (ordre croissant) sur ce tableau. Pour chaque passage, indique :

1. la valeur de l'indice i du passage ;
2. l'indice du minimum trouvé dans la partie non triée ;
3. l'état complet du tableau après l'échange éventuel.

Présente tes résultats dans un tableau à trois colonnes (Passage, indice du minimum, état du tableau).

EXERCICE 12 Exécution du tri par insertion

On considère le tableau d'entiers suivant, indicé à partir de 1 :

Indice	1	2	3	4	5
Valeur	9	3	12	5	20

1. Exécute pas à pas l'algorithme du tri par insertion (ordre croissant) : pour chaque élément inséré, donne l'état du tableau après insertion.
2. Compte le nombre total de décalages d'éléments effectués au cours du tri.

EXERCICE 13 Exécution d'une recherche dichotomique

On considère le tableau trié par ordre croissant, indicé à partir de 1 :

Indice	1	2	3	4	5	6	7
Valeur	2	5	8	13	21	34	55

On recherche la valeur 21 par l'algorithme de recherche dichotomique.

1. Recopie et complète le tableau de trace suivant, en donnant à chaque étape les valeurs de `debut`, `fin`, `milieu` et `T[milieu]` :

Étape	debut	fin	milieu	T[milieu]
1				
2				
3				

2. À quelle position la valeur 21 est-elle trouvée ? En combien d'étapes ?
3. Combien d'étapes aurait nécessitées une recherche séquentielle pour trouver cette même valeur ?

EXERCICE 14 Recherche séquentielle dans un tableau de noms

La bibliothèque du lycée de Bafoussam conserve dans un tableau `NOMS` de n chaînes de caractères les noms des élèves inscrits au club informatique.

Écris en langage algorithmique un algorithme qui :

- demande à l'utilisateur le nom d'un élève à rechercher ;
- parcourt le tableau `NOMS` séquentiellement ;
- affiche la position de l'élève dans le tableau s'il est trouvé, et le message « absent » sinon.

On supposera le tableau `NOMS` et sa taille n déjà connus de l'algorithme.

11.3 Je me prépare au Probatoire**EXERCICE 15** Tri des notes d'une classe de Première TI

À la fin du premier trimestre, le professeur d'informatique du lycée de Ngaoundéré saisit les notes de ses élèves dans un tableau `NOTES` de n réels ($n \leq 50$). Il souhaite les classer de la meilleure à la moins bonne.

1. Rappelle le principe du tri par sélection en trois phrases maximum.
2. Exécute le tri par sélection (ordre croissant) sur le tableau suivant, en donnant l'état du tableau après chaque passage :

Indice	1	2	3	4	5	6
Note	12	8,5	15	6	11	9

3. Écris en langage algorithmique un algorithme complet qui trie le tableau `NOTES` par ordre décroissant en adaptant le tri par sélection. Tu préciseras l'en-tête, la déclaration des variables et le corps de l'algorithme.
4. Le professeur veut ensuite afficher les trois meilleures notes de la classe. Explique, sans écrire d'algorithme, comment il peut exploiter le tableau trié par ordre décroissant pour obtenir immédiatement ce résultat.

EXERCICE 16 Chasse aux erreurs : un algorithme de recherche défaillant

Un élève de Première TI a écrit l'algorithme ci-dessous, censé rechercher la position d'une valeur x dans un tableau `T` de n entiers par recherche séquentielle. Malheureusement, son algorithme contient trois erreurs : une erreur de borne, une erreur d'incréméntation et une erreur de condition d'arrêt.

Algorithme RechercheEleve

Variable

T : Tableau[1..n] d'entiers

X, i, position : Entiers

trouve : Booleen

Debut

Lire(X)

i ← 0

trouve ← Faux

Tant que (i ≤ n) et (trouve = Faux) Faire

Si T[i] = X Alors

trouve ← Vrai

position ← i

FinSi

i ← i + 1

FinTant que

Si trouve = Vrai Alors

Ecrire("Valeur trouvee a la position ", position)

Sinon

Ecrire("Valeur absente")

FinSi

Fin

1. Recopie et complète le tableau suivant :

N°	Ligne fautive	Correction justifiée
1		
2		
3		

- Explique ce qui se passerait concrètement lors de l'exécution si l'erreur d'incréméntation n'était pas corrigée.
- Réécris l'algorithme complet et corrigé.
- Trace l'exécution de l'algorithme corrigé pour le tableau $T = [14 ; 7 ; 22 ; 9]$ avec $X = 22$, en donnant les valeurs successives de i et de $trouve$.

EXERCICE 17 Problème de synthèse : les matricules du concours

Le bureau des examens d'un collège d'examen de Yaoundé a enregistré dans un tableau MAT les numéros de matricule de 500 candidats, dans l'ordre d'arrivée des dossiers (le tableau n'est donc pas trié). Le jour de la publication des résultats, un candidat se présente et affirme être admis ; le surveillant doit vérifier rapidement si son matricule figure dans le tableau.

- Le surveillant décide d'effectuer une recherche séquentielle directement sur le tableau non trié. Combien de comparaisons devra-t-il effectuer dans le pire des cas ?

2. Un informaticien propose plutôt la démarche suivante : trier d'abord le tableau par ordre croissant, puis utiliser la recherche dichotomique. Explique pourquoi la recherche dichotomique est utilisable seulement après le tri.
3. Après tri, combien de comparaisons au maximum la recherche dichotomique effectuera-t-elle sur ce tableau de 500 éléments? Justifie ton calcul (on donne $2^8 = 256$ et $2^9 = 512$).
4. Écris en langage algorithmique l'algorithme de recherche dichotomique qui, à partir du tableau `MAT` trié et d'un matricule `M` saisi au clavier, affiche « Candidat admis : matricule trouvé à la position p » ou « Matricule introuvable ».
5. Le collègue devra effectuer des centaines de vérifications dans la journée. Justifie en trois ou quatre phrases que la démarche « trier une seule fois, puis rechercher dichotomiquement » est plus rentable que des recherches séquentielles répétées.

12 CORRIGÉS DES EXERCICES

13 LEÇON 04 : ALGORITHMES DE TRI ET DE RECHERCHE — CORRECTIONS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : tri et recherche

1. Réponse b). Le tri par sélection recherche, à chaque passage, le **plus petit élément** de la partie non triée et l'échange avec le premier élément de cette partie. La réponse a) décrit le tri par insertion, la réponse c) le tri à bulles, la réponse d) une stratégie de type « diviser pour régner ».
2. Réponse c). La condition d'application de la recherche dichotomique est que le tableau soit **trié** : c'est ce qui permet d'éliminer une moitié de la zone de recherche à chaque étape. Ni la taille, ni le type numérique (on peut trier des chaînes), ni l'absence de doublons ne sont exigés.
3. Réponse c). Dans le pire des cas (valeur absente ou en dernière position), la recherche séquentielle examine les n éléments un par un : n comparaisons.
4. Réponse b). Si la valeur cherchée est strictement supérieure à $T[\text{milieu}]$ dans un tableau trié par ordre croissant, elle ne peut se trouver que dans la moitié droite : on pose donc `debut ← milieu + 1`.

CORRIGÉ Exercice 2 — Vrai ou faux ? Justifie

1. Vrai. C'est exactement le principe du tri par insertion : la partie gauche du tableau est maintenue triée, et chaque nouvel élément y est inséré à sa place par décalages successifs (comme on trie des cartes en main).
2. Faux. La recherche séquentielle examine les éléments un à un dans l'ordre du tableau : elle fonctionne sur tout tableau, trié ou non. C'est la recherche dichotomique qui exige un tableau trié.
3. Vrai. À chaque passage, on place définitivement un élément à sa position finale. Après $n - 1$ passages, les $n - 1$ premiers éléments sont placés, et le dernier l'est automatiquement : il est inutile (et incorrect) de faire un n -ième passage.
4. Faux. Sur un tableau trié, la recherche dichotomique effectue au plus environ $\log_2 n$ comparaisons contre n pour la recherche séquentielle : elle est donc bien plus rapide dès que n grandit. (Nuance : si le tableau n'est pas trié, la dichotomie est inapplicable, et le tri préalable a un coût.)

CORRIGÉ Exercice 3 — Définitions

1. Algorithme de tri : suite finie et ordonnée d'opérations permettant de ranger les éléments d'un ensemble de données (par exemple un tableau) selon un ordre déterminé (croissant ou décroissant) défini par une clef de comparaison.
2. Recherche séquentielle : méthode de recherche qui consiste à parcourir le tableau élément par élément, depuis le premier, en comparant chacun à la valeur cherchée, jusqu'à trouver celle-ci ou épuiser le tableau. Elle ne suppose aucune condition sur l'ordre des éléments.
3. Recherche dichotomique : méthode de recherche applicable uniquement à un tableau trié, qui compare la valeur cherchée à l'élément du milieu de la zone de recherche, puis élimine la moitié de cette zone où la valeur ne peut pas se trouver, et recommence jusqu'à trouver la valeur ou réduire la zone à vide.
4. Clef de tri : donnée (ou champ) de chaque élément sur laquelle porte la comparaison lors du tri. Par exemple, pour trier des élèves, la clef peut être la note, le nom ou le matricule.

CORRIGÉ Exercice 4 — Calculs directs

1. Dans le pire des cas, la recherche séquentielle compare la valeur cherchée aux n éléments : pour $n = 10$, cela fait 10 comparaisons.
2. À chaque étape, la zone de recherche est divisée par 2. En partant de $128 = 2^7$ éléments, on obtient successivement des zones de taille 128, 64, 32, 16, 8, 4, 2, 1 : il faut donc au maximum 7 étapes (comparaisons) pour réduire la zone à un seul élément et conclure. De façon générale, le maximum est $\log_2 n$ lorsque n est une puissance de 2.
3. On cherche le plus petit entier k tel que $2^k \geq 1000$. Comme $2^9 = 512 < 1000$ et $2^{10} = 1024 \geq 1000$, il faut au maximum 10 étapes. La recherche séquentielle, elle, exigerait jusqu'à 1000 comparaisons dans le pire des cas : la dichotomie est ici environ 100 fois plus économe.

CORRIGÉ Exercice 5 — Exécution du tri par sélection

Tableau initial : [25; 13; 40; 7; 18].

Passage $i = 1$: on cherche le minimum dans $T[1..5] = [25; 13; 40; 7; 18]$. Le minimum est 7, à l'indice 4. On échange $T[1]$ et $T[4]$: [7; 13; 40; 25; 18].

Passage $i = 2$: minimum de $T[2..5] = [13; 40; 25; 18]$: c'est 13, indice 2. Aucun échange nécessaire : [7; 13; 40; 25; 18].

Passage $i = 3$: minimum de $T[3..5] = [40; 25; 18]$: c'est 18, indice 5. Échange de $T[3]$ et $T[5]$:

[7; 13; 18; 25; 40].

Passage $i = 4$: minimum de $T[4..5] = [25; 40]$: c'est 25, indice 4. Aucun échange : [7; 13; 18; 25; 40].

Passage i	Indice du minimum	État du tableau après le passage
1	4	[7; 13; 40; 25; 18]
2	2	[7; 13; 40; 25; 18]
3	5	[7; 13; 18; 25; 40]
4	4	[7; 13; 18; 25; 40]

Le tableau est trié : [7; 13; 18; 25; 40]. On constate qu'il y a bien eu $n - 1 = 4$ passages.

CORRIGÉ Exercice 6 — Exécution du tri par insertion

Tableau initial : [9; 3; 12; 5; 20]. La partie triée initiale est [9].

- Insertion de $T[2] = 3$: $3 < 9$, on décale 9 et on insère 3 en position 1 : [3; 9; 12; 5; 20]. (1 décalage)
 Insertion de $T[3] = 12$: $12 > 9$, il est déjà à sa place : [3; 9; 12; 5; 20]. (0 décalage)
 Insertion de $T[4] = 5$: $5 < 12$ (décalage de 12), $5 < 9$ (décalage de 9), $5 > 3$: on insère 5 en position 2 : [3; 5; 9; 12; 20]. (2 décalages)
 Insertion de $T[5] = 20$: $20 > 12$, déjà à sa place : [3; 5; 9; 12; 20]. (0 décalage)
- Nombre total de décalages : $1 + 0 + 2 + 0 = 3$ décalages.

Le tableau trié est [3; 5; 9; 12; 20].

CORRIGÉ Exercice 7 — Exécution d'une recherche dichotomique

- Tableau de trace pour la recherche de 21 :
 Étape 1 : $\text{debut} = 1, \text{fin} = 7, \text{milieu} = (1 + 7) \div 2 = 4, T[4] = 13$. Comme $21 > 13$, on poursuit à droite : $\text{debut} \leftarrow 5$.
 Étape 2 : $\text{debut} = 5, \text{fin} = 7, \text{milieu} = (5 + 7) \div 2 = 6, T[6] = 34$. Comme $21 < 34$, on poursuit à gauche : $\text{fin} \leftarrow 5$.
 Étape 3 : $\text{debut} = 5, \text{fin} = 5, \text{milieu} = 5, T[5] = 21$: valeur trouvée.

Étape	debut	fin	milieu	T[milieu]
1	1	7	4	13
2	5	7	6	34
3	5	5	5	21

- La valeur 21 est trouvée à la position 5, en 3 étapes.
- En recherche séquentielle, on aurait examiné $T[1], T[2], \dots, T[5]$: il aurait fallu 5 comparaisons. La dichotomie est ici plus rapide, et l'écart grandit avec la taille du tableau.

CORRIGÉ Exercice 8 — Recherche séquentielle dans un tableau de noms

Algorithme RechercheEleveClub

Variable

NOMS : Tableau[1..n] de chaînes

nomCherche : Chaîne

i, position : Entiers

trouve : Booleen

Debut

Ecrire("Entrez le nom de l'élève à rechercher : ")

Lire(nomCherche)

i ← 1

trouve ← Faux

Tant que (i ≤ n) et (trouve = Faux) Faire

Si NOMS[i] = nomCherche Alors

trouve ← Vrai

position ← i

FinSi

i ← i + 1

FinTant que

Si trouve = Vrai Alors

Ecrire("Eleve trouve a la position ", position)

Sinon

Ecrire("absent")

FinSi

Fin

Points de vigilance : initialiser i à 1 (tableau indicé à partir de 1) ; la condition $i \leq n$ garantit qu'on ne dépasse pas la taille du tableau ; la condition $\text{trouve} = \text{Faux}$ permet d'arrêter le parcours dès que l'élève est trouvé, ce qui évite des comparaisons inutiles.

CORRIGÉ Exercice 9 — Tri des notes d'une classe de Première TI

- Principe du tri par sélection : on parcourt le tableau de la gauche vers la droite. À chaque passage i, on recherche le plus petit élément de la partie non triée $T[i..n]$. On

échange ensuite cet élément avec $T[i]$, ce qui agrandit d'une case la partie triée.

2. Exécution (ordre croissant) sur [12; 8,5; 15; 6; 11; 9] :

Passage i	Indice du min	État du tableau après le passage
1	4	[6; 8,5; 15; 12; 11; 9]
2	2	[6; 8,5; 15; 12; 11; 9]
3	6	[6; 8,5; 9; 12; 11; 15]
4	5	[6; 8,5; 9; 11; 12; 15]
5	5	[6; 8,5; 9; 11; 12; 15]

Détail : passage 1, min de [12; 8,5; 15; 6; 11; 9] = 6 (indice 4), échange $T[1] \leftrightarrow T[4]$; passage 2, min de [8,5; 15; 12; 11; 9] = 8,5 (indice 2), pas d'échange; passage 3, min de [15; 12; 11; 9] = 9 (indice 6), échange $T[3] \leftrightarrow T[6]$; passage 4, min de [12; 11; 15] = 11 (indice 5), échange $T[4] \leftrightarrow T[5]$; passage 5, min de [12; 15] = 12 (indice 5), pas d'échange. Tableau trié : [6; 8,5; 9; 11; 12; 15].

3. Algorithme de tri par sélection en ordre décroissant (on recherche le maximum au lieu du minimum) :

```

Algorithme TriNotesDecroissant
Constante
    NMAX = 50
Variable
    NOTES : Tableau[1..NMAX] de reels
    n, i, j, indiceMax : Entiers
    temp : Reel
Debut
    Ecrire("Nombre d'eleves (<= 50) : ")
    Lire(n)
    Pour i de 1 a n Faire
        Ecrire("Note numero ", i, " : ")
        Lire(NOTES[i])
    FinPour
    Pour i de 1 a n - 1 Faire
        indiceMax <- i
        Pour j de i + 1 a n Faire
            Si NOTES[j] > NOTES[indiceMax] Alors
                indiceMax <- j
            FinSi
        FinPour
        Si indiceMax <> i Alors
            temp <- NOTES[i]
            NOTES[i] <- NOTES[indiceMax]
            NOTES[indiceMax] <- temp
        FinSi
    FinPour
    Ecrire("Notes triees de la meilleure a la moins bonne :")
    Pour i de 1 a n Faire
        Ecrire(NOTES[i])
    FinPour
Fin
  
```

4. Une fois le tableau trié par ordre décroissant, les trois meilleures notes occupent automatiquement les trois premières cases : il suffit d'afficher `NOTES[1]`, `NOTES[2]` et `NOTES[3]`, sans aucune recherche supplémentaire. C'est tout l'intérêt pratique du tri : une fois les données ordonnées, de nombreuses questions (meilleurs éléments, médiane, recherche rapide) deviennent immédiates.

Barème indicatif (type Probatoire, sur 10 points) : principe du tri : 2 pts ; exécution correcte des 5 passages : 3 pts ; algorithme (en-tête et déclarations 1 pt, double boucle avec maximum 2 pts, échange 1 pt) : 5 pts.

Points de vigilance : pour l'ordre décroissant, c'est bien le maximum qu'on recherche (comparaison `>`) ; la boucle principale s'arrête à $n - 1$; l'échange exige une variable temporaire `temp`, sans laquelle une valeur serait écrasée.

CORRIGÉ Exercice 10 — Chasse aux erreurs : un algorithme de recherche défaillant

1. Tableau des erreurs :

N°	Ligne fautive	Correction justifiée
1	<code>i <- 0</code> (erreur de borne)	<code>i <- 1</code> : le tableau est indicé de 1 à n ; avec <code>i = 0</code> , la première comparaison porte sur <code>T[0]</code> qui n'existe pas.
2	<code>i <- i</code> (erreur d'incrémentation)	<code>i <- i + 1</code> : sans progression de l'indice, on examine toujours la même case.
3	Tant que (<code>i <= n</code>) ... combiné à l'absence d'incrément correcte (condition d'arrêt)	Avec l'incrément corrigée, la condition (<code>i <= n</code>) et (<code>trouve = Faux</code>) est correcte ; l'erreur de condition d'arrêt réside dans le fait que la boucle ne peut jamais se terminer tant que <code>X</code> n'est pas en <code>T[i]</code> : la correction de la ligne 2 rétablit l'arrêt. On veillera donc à conserver la double condition (<code>i <= n</code>) et (<code>trouve = Faux</code>).

Remarque : l'énoncé regroupe les trois erreurs classiques : borne initiale (`i <- 0`), incrément (`i <- i`), et condition d'arrêt rendue inopérante par l'absence d'incrément. La correction des lignes 1 et 2 rend la condition d'arrêt effective.

2. Si l'erreur d'incrément n'est pas corrigée, la variable `i` garde indéfiniment sa valeur initiale. Deux cas se présentent : si `T[i] = X`, la boucle s'arrête (par chance) ; sinon, la condition `i <= n` reste vraie pour toujours et `trouve` reste `Faux` : le programme

tourne sans fin. C'est une **boucle infinie** : l'ordinateur ne répond plus et le programme doit être interrompu de force.

3. Algorithme corrigé :

```

Algorithme RechercheSequentielleCorrige
Variable
  T : Tableau[1..n] d'entiers
  X, i, position : Entiers
  trouve : Booleen
Debut
  Lire(X)
  i ← 1
  trouve ← Faux
  Tant que (i ≤ n) et (trouve = Faux) Faire
    Si T[i] = X Alors
      trouve ← Vrai
      position ← i
    FinSi
    i ← i + 1
  FinTant que
  Si trouve = Vrai Alors
    Ecrire("Valeur trouvée à la position ", position)
  Sinon
    Ecrire("Valeur absente")
  FinSi
Fin
  
```

4. Trace pour $T = [14 ; 7 ; 22 ; 9]$ et $X = 22$:

Tour	i (test)	$T[i]$	trouve
1	1	14	Faux
2	2	7	Faux
3	3	22	Vrai

La boucle s'arrête car `trouve = Vrai`; l'algorithme affiche « Valeur trouvée à la position 3 ».

Barème indicatif (sur 10 points) : identification des trois erreurs : 3 pts ; explication de la boucle infinie : 2 pts ; algorithme corrigé : 3 pts ; trace d'exécution : 2 pts.

Points de vigilance : toute boucle `Tant que` doit contenir une instruction qui fait progresser sa condition vers l'arrêt ; toujours vérifier que les indices restent dans l'intervalle $[1; n]$; tester mentalement l'algorithme sur un petit exemple (comme à la question 4) est la meilleure façon de débusquer ces erreurs.

CORRIGÉ

Exercice 11 — Problème de synthèse : les matricules du concours

1. Dans le pire des cas (matricule absent ou en dernière position), la recherche séquentielle examine tous les éléments : 500 comparaisons.
2. La recherche dichotomique repose sur l'élimination d'une moitié de la zone de recherche à chaque étape : après comparaison avec l'élément du milieu, on doit être certain que la valeur cherchée ne peut pas se trouver dans la moitié abandonnée. Cette certitude n'existe que si les éléments sont ordonnés : si $M > \text{MAT}[\text{milieu}]$ dans un tableau trié par ordre croissant, alors M est strictement supérieur à tous les éléments de la moitié gauche. Sur un tableau non trié, rien ne garantit cela : la dichotomie pourrait écarter la zone contenant la valeur cherchée et conclure à tort à son absence.
3. On cherche le plus petit entier k tel que $2^k \geq 500$. Comme $2^8 = 256 < 500$ et $2^9 = 512 \geq 500$, il faut au maximum 9 comparaisons. En effet, après 8 divisions par 2, la zone contient encore $\lceil 500/256 \rceil = 2$ éléments ; une 9^{ème} comparaison permet de conclure.
4. Algorithme de recherche dichotomique :

```

Algorithme VerificationMatricule
Variable
  MAT : Tableau[1..500] d'entiers
  M, debut, fin, milieu, position : Entiers
  trouve : Booleen
Debut
  Ecrire("Entrez le matricule du candidat : ")
  Lire(M)
  debut <- 1
  fin <- 500
  trouve <- Faux
  Tant que (debut <= fin) et (trouve = Faux) Faire
    milieu <- (debut + fin) div 2
    Si MAT[milieu] = M Alors
      trouve <- Vrai
      position <- milieu
    Sinon
      Si M > MAT[milieu] Alors
        debut <- milieu + 1
      Sinon
        fin <- milieu - 1
    FinSi
  FinSi
  FinTant que
  Si trouve = Vrai Alors
    Ecrire("Candidat admis : matricule trouve a la position ", position)
  Sinon
    Ecrire("Matricule introuvable")
  FinSi
Fin

```

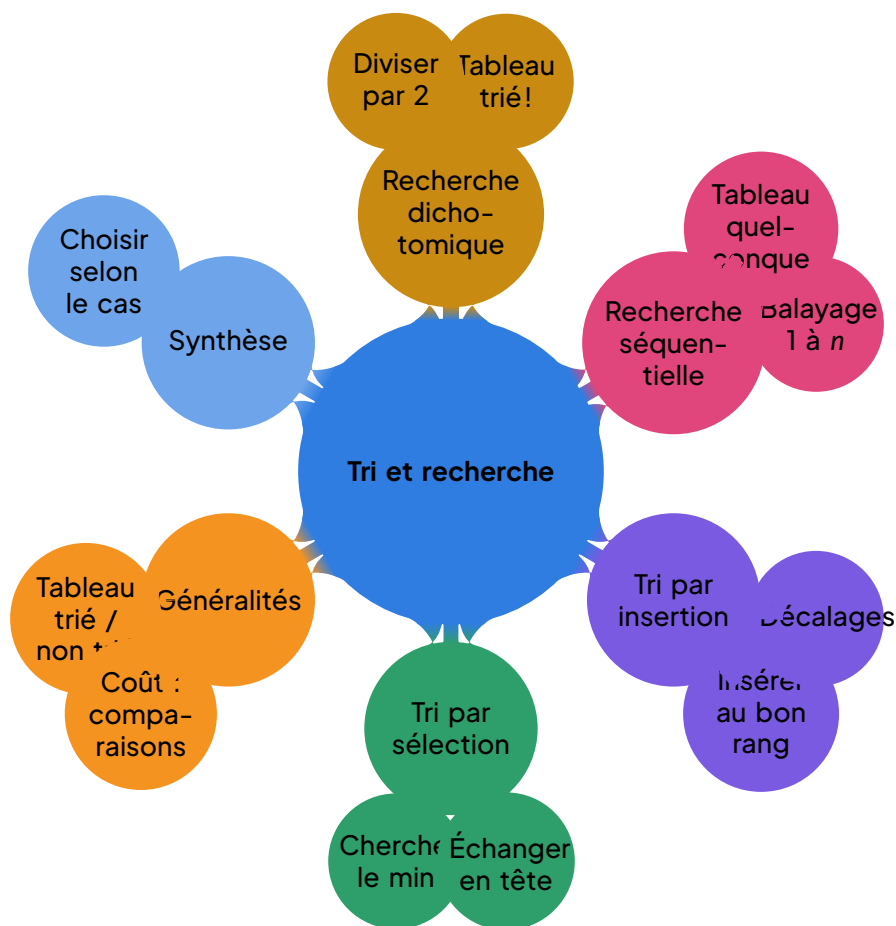
5. Le tri du tableau n'est effectué qu'une seule fois, au début de la journée. Ensuite, chaque vérification coûte au plus 9 comparaisons par dichotomie, contre jusqu'à 500 par recherche séquentielle. Pour des centaines de vérifications, on passe ainsi de plusieurs centaines de milliers de comparaisons à quelques milliers, auxquelles s'ajoute le coût unique du tri. Le temps d'attente de chaque candidat est donc considérable-

ment réduit : l'investissement initial du tri est largement amorti.

Barème indicatif (type Probatoire, sur 10 points) : question 1 : 1 pt ; question 2 (justification de la condition « tableau trié ») : 2 pts ; question 3 (calcul avec 2^8 et 2^9) : 2 pts ; question 4 (algorithme complet et correct) : 3 pts ; question 5 (argumentation chiffrée) : 2 pts.

Points de vigilance : citer explicitement la condition d'application de la dichotomie (tableau trié) dès qu'on l'utilise ; le calcul du milieu utilise la division entière (`div`) ; les bornes se mettent à jour par `debut ← milieu + 1` et `fin ← milieu - 1` (et non `milieu`, sinon risque de boucle infinie) ; la condition de continuation est `debut <= fin`.

14 FICHE BILAN



Carte mentale de la leçon : trier un tableau, puis y rechercher un élément efficacement.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Trier** un tableau : ranger ses éléments dans un ordre croissant (ou décroissant).
- **Tri par sélection** : à chaque passage, on cherche le minimum de la partie non triée et on l'échange avec le premier élément de cette partie.
- **Tri par insertion** : on prend chaque élément et on l'insère à sa place dans la partie déjà triée, en décalant les éléments plus grands.
- **Recherche séquentielle** : on parcourt le tableau du premier au dernier élément jusqu'à trouver la valeur cherchée (ou épuiser le tableau).
- **Recherche dichotomique** : dans un tableau trié, on compare avec l'élément du milieu

et on élimine la moitié du tableau à chaque étape.

Complexités à connaître par cœur

Algorithme	Nombre d'opérations	Condition d'usage
Tri par sélection	$n(n-1)/2$ comparaisons ($\approx n^2$)	Aucune (tableau quelconque)
Tri par insertion	au pire $n(n-1)/2$ comparaisons	Efficace si presque trié
Recherche séquentielle	au pire n comparaisons	Aucune
Recherche dichotomique	au pire $\log_2(n)+1$ comparaisons	Tableau trié obligatoire

Méthodes express

- Sélection : pour i de 1 à $n-1$, trouver l'indice $i_{\min}$ du min entre i et n , puis échanger $T[i]$ et $T[i_{\min}]$.
- Insertion : pour i de 2 à n , mémoriser $x \leftarrow T[i]$, décaler vers la droite les éléments $> x$, placer x dans le trou.
- Séquentielle : $i \leftarrow 1$; tant que $i \leq n$ et $T[i] \neq x$, $i \leftarrow i+1$; trouvé si $i \leq n$.
- Dichotomique : bornes $g \leftarrow 1$, $d \leftarrow n$; tant que $g \leq d$: $m \leftarrow (g+d) \text{ div } 2$; si $T[m] = x$ trouvé; si $T[m] < x$ alors $g \leftarrow m+1$ sinon $d \leftarrow m-1$.
- Échange de deux cases : toujours passer par une variable temporaire.

À RETENIR Checklist avant le Bac

- ☐ Je sais définir le tri et la recherche dans un tableau.
- ☐ Je sais exécuter à la main un tri par sélection (trace des échanges).
- ☐ Je sais exécuter à la main un tri par insertion (trace des décalages).
- ☐ Je sais écrire l'algorithme complet d'un tri simple.
- ☐ Je n'oublie jamais la variable temporaire pour échanger deux cases.
- ☐ Je sais décrire le principe de la recherche séquentielle.
- ☐ Je sais écrire l'algorithme de recherche d'un élément dans un tableau.
- ☐ Je sais exécuter une recherche dichotomique étape par étape (calcul de m , bornes g et d).
- ☐ Je me souviens que la dichotomie exige un tableau trié.
- ☐ Je connais les coûts : n^2 pour les tris simples, n pour la séquentielle, $\log_2 n$ pour la dichotomie.

- ☐ Je fais attention aux bornes des boucles (1 à $n-1$ pour la sélection, 2 à n pour l'insertion).
- ☐ Je sais choisir l'algorithme adapté à la situation (tableau trié ou non, taille des données).

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+